LCC PROTOCOL · SECURITY REVIEW

LCC Protocol Review.

A security review of 3Jane's LCC protocol.

CLIENT	PROJECT	REVIEW FIRM	REVIEW WINDOW	FINAL REPORT
3Jane	LCC Protocol	Guardian	August 3 - August 26, 2026	August 27, 2026

§ 00

Contents.

70 pages - overview, confidence ranking, scope, 54 findings, disclaimer, and about Guardian.

§ 01 **Overview** p. 03

§ 02 **Confidence Ranking** p. 04

§ 03 **Scope & Classifications** p. 05

Files in Scope p. 06

§ 04 **Findings** p. 07

Round 1 - Main Review · 44 findings p. 12-56

Round 2 - Remediation Review · 10 findings p. 57-68

§ 05 **Disclaimer** p. 69

§ 06 **About Guardian** p. 70

§ 01

Overview.

Guardian reviewed 3Jane's LCC protocol through a main review followed by remediation and update validation.

REVIEW FIRM	Guardian
CLIENT	3Jane
PROJECT	LCC Protocol
PREPARED BY	0xCiphky · Wafflemakr
MAIN REVIEW	August 3 - August 11, 2026
REMEDIATION & UPDATES	August 20 - August 26, 2026
FINAL REPORT	August 27, 2026

REPOSITORY	https://github.com/3jane-protocol/moneymarket-contracts
MAIN REVIEW COMMIT	`777accc692f1a87e2ba1367695cacd15a5a5e8a8`
REMEDIATION COMMIT	`74353c0564755da516c2bd7892fbbfeede38e082`

1.1 Confidence Ranking

4/5

High Confidence

The review covered the consolidated LCC protocol scope across main-review and remediation validation stages. No High or Critical findings remain in the consolidated report.

1	2	3	**4**	5
V. LOW	LOW	MED	**HIGH**	V. HIGH

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	IN PROG	DECLINED	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0	0	0
High	0	0	0	0	0	0	0
Medium	**20**	0	0	0	**12**	**2**	**6**
Low	**31**	0	0	0	**24**	0	**7**
Info	**3**	0	0	0	**1**	0	**2**

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3 Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas such as weak modularity or risky patterns. No critical design flaws were identified, though some patterns could lead to edge-case issues. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2 Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week, or additional unresolved High/Critical findings uncovered in remediation review.
1 Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (>=5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	>=5 High/Critical findings and overall systemic flaws.

§ 03

Scope & Classifications.

3.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

§ 03

Scope.

Files in scope, grouped by exact repository path. Source-line counts reproduce the supplied export.

moneymarket-contracts

14 FILES · 1,947 MEASURED SLOC

```
moneymarket-contracts/src/usd3/
  NotificationVault.sol                                  107
  USD3.sol                                               379
moneymarket-contracts/src/lcc/
  LCCVault.sol                                          1004
  LCCVaultFactory.sol                                     34
moneymarket-contracts/src/libraries/
  ConstantsLib.sol                                         9
  ProtocolConfigLib.sol                                   29
moneymarket-contracts/src/lcc/libraries/
  LCCAccountLib.sol                                       51
  LCCAuctionLib.sol                                      100
  LCCBucketListLib.sol                                    20
  LCCConfigLib.sol                                        57
  LCCErrorsLib.sol                                        33
  LCCEventsLib.sol                                        47
  LCCTypesLib.sol                                         77
  LCCExitLib.sol
```

§ 04

Findings.

54 findings across two review stages: 20 Medium findings, 31 Low findings, 3 Informational findings.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
M-01	Debt-Funded Shares Fake Junior Backing	Logical Error	MEDIUM	ACKNOWLEDGED	p. 12
M-02	Cured Markdowns Transfer Junior Principal	Logical Error	MEDIUM	ACKNOWLEDGED	p. 13
M-03	Pending Losses Inflate Credit Deployment Cap	Logical Error	MEDIUM	RESOLVED	p. 14
M-04	Fee Shares Recapture Their Own Profit	Logical Error	MEDIUM	ACKNOWLEDGED	p. 15
M-05	Slash rake is capped by the leftover	Logical Error	MEDIUM	RESOLVED	p. 16
M-06	Impaired tranche makes LCC default free	Logical Error	MEDIUM	RESOLVED	p. 17
M-07	Stale Markdown Lets Early USD3 Holders Exit	Logical Error	MEDIUM	ACKNOWLEDGED	p. 18
M-08	Exact-floor settlement forges USD3 NAV that extracts fresh depositor liquidity	Gaming	MEDIUM	PARTIALLY RESOLVED	p. 19
M-09	Tranche fee mint inflates the borrow cap	Logical Error	MEDIUM	ACKNOWLEDGED	p. 20
M-10	Tranche Changes Reprice Accrued Profit	Logical Error	MEDIUM	ACKNOWLEDGED	p. 21
M-11	Shutdown exits remain coupled to a failed high-utilization Morpho valuation	DoS	MEDIUM	ACKNOWLEDGED	p. 22

§ 04

Findings, continued.

Index page 2 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
M-12	waUSDC pauses can block willing LCC funders and expose their entire margin to auction	Informational	MEDIUM	RESOLVED	p. 23
M-13	Backdated zero-to-nonzero DRP creates redeemable phantom USD3 yield	Informational	MEDIUM	PARTIALLY RESOLVED	p. 24
M-14	Wind-down report rollback lets a cured borrower cash out junior capital and shift its next default to seniors	Informational	MEDIUM	RESOLVED	p. 25
M-15	Markdown-excluded deployment accounting lets cured borrowers drain reserves and realize losses above MAX_ON_CREDIT	Informational	MEDIUM	ACKNOWLEDGED	p. 26
M-16	Attacker-chosen premium crystallization lets new USD3 shares cash out historical unpaid yield	Informational	MEDIUM	ACKNOWLEDGED	p. 27
M-17	Permissionless cures let late USD3 depositors snipe senior recovery	Logical Error	MEDIUM	ACKNOWLEDGED	p. 28
M-18	Junior holders can withdraw fees on unpaid income before default	Informational	MEDIUM	ACKNOWLEDGED	p. 29
L-01	Temporary Deposits Bypass MAX_ON_CREDIT	Logical Error	LOW	ACKNOWLEDGED	p. 30
L-02	Beacon upgrade bricks vault allowances	Upgradeability	LOW	ACKNOWLEDGED	p. 31
L-03	Blocked live exiter has no funding path	DoS	LOW	ACKNOWLEDGED	p. 32
L-04	sUSD3 entry misses USD3 loss guard	Validation	LOW	ACKNOWLEDGED	p. 33

§ 04

Findings, continued.

Index page 3 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
L-05	Unstake reverts on a paused wrapper	DoS	LOW	RESOLVED	p. 34
L-06	Revoked Bypass Reuses Old Cooldown	Logical Error	LOW	RESOLVED	p. 35
L-07	Transferred Shares Bypass Minimum Deposit	Validation	LOW	ACKNOWLEDGED	p. 36
L-08	Slash Sync Defeats Auction Off-Switch	Logical Error	LOW	ACKNOWLEDGED	p. 37
L-09	Zero Unlock Bypasses USD3 Loss Buffer	Logical Error	LOW	RESOLVED	p. 38
L-10	Premium accrual can brick a borrower	Math	LOW	RESOLVED	p. 39
L-11	Fee changes retroactively reallocate realized borrower premiums	Informational	LOW	ACKNOWLEDGED	p. 40
L-12	High USD3 PPS magnifies first-loss floor rounding into material senior losses	Informational	LOW	ACKNOWLEDGED	p. 41
L-13	Unlimited DEBT_CAP overflows waUSDC conversion and blocks sUSD3 deposits	Informational	LOW	ACKNOWLEDGED	p. 42
L-14	Share-floor phantom liquidity makes USD3 loss reports revert and bricks deposits	Informational	LOW	ACKNOWLEDGED	p. 43
L-15	USD3 maxDeposit and maxMint advertise cap fragments rejected by minDeposit	Informational	LOW	RESOLVED	p. 44

§ 04

Findings, continued.

Index page 4 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
L-16	One defaulted borrower freezes exits	DoS	LOW	ACKNOWLEDGED	p. 45
L-17	Funding depends on an unbound USD3 flag	Validation	LOW	RESOLVED	p. 46
L-18	Consumed junior backing remains borrowable when USD3 cannot rebalance	Informational	LOW	ACKNOWLEDGED	p. 48
L-19	One-share sUSD3 inflation steals up to 25% from the next junior depositor	Informational	LOW	ACKNOWLEDGED	p. 49
L-20	Latent borrower premiums let fresh deposits bypass deployment and debt caps	Informational	LOW	ACKNOWLEDGED	p. 50
L-21	Matured exit claims accrue irrelevant replay debt from later calls	Informational	LOW	RESOLVED	p. 51
L-22	Pre-report tranche switching captures historical junior fees	Informational	LOW	ACKNOWLEDGED	p. 52
L-23	Stale waUSDC capacity quote makes USD3 repeatedly retry an unexecutable exact maximum	Informational	LOW	ACKNOWLEDGED	p. 53
L-24	Yearn protocol fee reduces the configured sUSD3 yield share	Informational	LOW	ACKNOWLEDGED	p. 54
L-25	Direct USD3 transfers let fresh junior capital evade its lock and shift default losses to seniors	Informational	LOW	ACKNOWLEDGED	p. 55
L-26	Locked profit bypasses junior loss waterfall	Logical Error	LOW	ACKNOWLEDGED	p. 56

§ 04

Findings, continued.

Index page 5 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 2 - REMEDIATION REVIEW					
M-01	Dust repayment backdates premium to a later draw	Logical Error	MEDIUM	RESOLVED	p. 57
M-02	Gross Interest on Marked Debt Becomes Yield	Logical Error	MEDIUM	ACKNOWLEDGED	p. 59
L-01	Recapitalization hides later settlement losses	Logical Error	LOW	ACKNOWLEDGED	p. 60
L-02	Disabled Markdown Leaves Stale Exit Window	Logical Error	LOW	ACKNOWLEDGED	p. 62
L-03	Fee Preview Leaves Deployment Above Cap	Logical Error	LOW	ACKNOWLEDGED	p. 63
L-04	Minimum Credit-Line Check Blocks Emergency Revocation	Logical Error	LOW	ACKNOWLEDGED	p. 64
L-05	Minimum Debt Check Prevents Terminal Insurance Use	Logical Error	LOW	ACKNOWLEDGED	p. 65
I-01	Activation gates live only in NatSpec	Documentation	INFO	RESOLVED	p. 66
I-02	Aave index cast can freeze the market	DoS	INFO	ACKNOWLEDGED	p. 67
I-03	Vault creation does not bind oracle to asset	Validation	INFO	RESOLVED	p. 68

MEDIUM M-01 ACKNOWLEDGED

Debt-Funded Shares Fake Junior Backing.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L184-L203 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 treats every USD3 share held by sUSD3 as independent first-loss capital and permits deployment up to that value divided by MIN_SUSD3_BACKING_RATIO. It does not distinguish genuine backing from shares purchased using an unpaid loan from the same Morpho market. An approved borrower can borrow USDC and deposit through a debt-free receiver because availableDepositLimit() checks the receiver rather than the payer or funding source. The receiver can transfer the resulting USD3 directly to sUSD3, increasing _subordinationDeployCapWaUSDC() while bypassing sUSD3's deposit cap, whitelist, lock and tranche-ratio checks. No keeper, LCC or honest deposit is required. Assume 10,000 USDC of senior capital, 100 USDC of genuine junior backing, a 10% ratio and the live 1,000-USDC minimum deposit. Genuine backing initially supports 1,000 of debt. The borrower draws and deposits that 1,000 through a clean receiver, transfers 999.999999 USD3 to sUSD3 while leaving one raw share unit, then deposits enough USDC to mint one raw share unit (two USDC base units at the observed live share price). The minimum no longer applies because the receiver holds a share. TokenizedStrategy therefore invokes deployFunds() with USD3's entire loose balance, increasing backing to ~1,100 and supported debt to ~11,000. The borrower extracts another 10,000 before default. Without recycling, default on the initial 1,000 debt burns the genuine backing and causes ~900 of senior loss. After recycling, the 1,100 of genuine and circular backing is burned, but the circular portion was purchased with unpaid debt. ~100 remains against the original 10,000 senior position, causing ~9,900 of senior loss and approximately 9,000 of additional loss. At exact 100% utilization, loss reporting may revert while attempting to withdraw Morpho's residual supply-share price floor if no token liquidity remains; the borrower can avoid this by leaving a negligible amount of waUSDC in Morpho before settlement. For recycled amount x, the cap increases by x/r after debt already increased by x, creating x * (1/r - 1) of uncovered senior exposure whenever r < 100%. MIN_SUSD3_BACKING_RATIO is intended to ensure that borrower debt remains covered by subordinate capital. Because the cap measures sUSD3's token balance rather than economically independent principal, debt-funded shares satisfy the same constraint they are meant to secure. The invariant can therefore be bypassed whenever the configured ratio is below 100%.

RECOMMENDATION

Do not derive the lending/deployment cap directly from sUSD3's transferable USD3 balance. Use a non-circular eligible-backing limit established before borrower drawdown and reduced on withdrawals and losses. If independent capital cannot be established, require 100% backing or segregate junior funding from the protected lending market. Enforce totalBorrowAssets <= eligibleBacking / backingRatio after all interest, premium and loss-accounting updates.

RESOLUTION

Status: Acknowledged.

MEDIUM M-02 ACKNOWLEDGED

Cured Markdowns Transfer Junior Principal.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L567-L604 REVIEW Round 1 - Main Review

DESCRIPTION

A temporary MorphoCredit markdown permanently burns sUSD3-owned principal, but a later cure is treated as new profit instead of restoring the junior tranche that absorbed the loss. Fully reversible credit marks can therefore transfer value from junior holders to senior USD3 holders without any realized loan write-off. When USD3 reports a markdown loss, it first consumes remaining locked-profit shares and then permanently burns enough USD3 shares from sUSD3 to cover the residual loss. USD3 records no junior-loss carry identifying how many shares or assets sUSD3 absorbed. MorphoCredit markdowns are reversible. If the borrower satisfies its overdue obligation, its status becomes Current, the calculated markdown returns to zero, and MorphoCredit adds the prior markdown back to totalSupplyAssets. The next USD3 report sees this restored loan value as ordinary profit. It mints only the configured tranche-share fee to sUSD3 and locks the remaining profit for every surviving USD3 holder; it does not first restore the burned junior principal. For example, let USD3 begin with 100 assets and 100 shares: seniors own 85 shares and sUSD3 owns 15. A reported 10-asset markdown burns 10 of sUSD3's shares, leaving 90 assets, 90 shares, seniors still worth 85, and sUSD3 worth 5. If the borrower cures and restores the 10 assets, a 15% tranche share mints 1.5 fee shares to sUSD3 and locks the other 8.5 shares. After full unlock, USD3 has 100 assets and 91.5 effective shares. Seniors are worth approximately 92.896 while sUSD3 is worth only 7.104. Although the loan value fully recovered, approximately 7.896 moved permanently from junior to senior. The issue does not require sUSD3 to be wiped out or insolvent: the markdown can be smaller than junior backing, seniors can absorb none of the original loss, and the tranche remains operational. Any caller can materialize borrower markdown, while the borrower or a third-party payer controls when the obligation is cured. Normal keeper reports complete both sides of the transfer. Repeated default-and-cure cycles can repeatedly erode junior NAV; borrower penalties and JANE slashing do not reconcile the USD3/sUSD3 accounting imbalance.

RECOMMENDATION

Track recovery carries for every layer of the loss waterfall. A loss absorbed by sUSD3 should create a junior-loss carry, while any uncovered amount should create a senior-loss carry. When a markdown reverses or identified principal is recovered, restore outstanding carries in reverse loss order before applying ordinary profit locking or tranche fees.

RESOLUTION

Status: Acknowledged.

MEDIUM M-03 RESOLVED

Pending Losses Inflate Credit Deployment Cap.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L182-L209 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 can deploy additional liquidity against junior backing that a materialized but unreported loss will consume. A second approved borrower can atomically trigger this deployment and borrow the liquidity before USD3 reports the loss. USD3 calculates its subordination cap by valuing sUSD3's USD3 shares using TokenizedStrategy's stored totalAssets and totalSupply. When a borrower markdown or settlement reduces Morpho's totalSupplyAssets, USD3's live Morpho claim and nav() fall immediately, but its stored accounting and the calculated junior value remain unchanged until report(). During this interval, _deployFunds() and tend() interpret the reduced Morpho claim as deployment headroom while targeting the stale, higher cap. They can therefore supply existing local waUSDC against junior backing that the next report will burn. A debt-free contract with an approved credit line can exploit this atomically. It calls the permissionless accruePremiumsForBorrowers() to materialize another borrower's markdown, makes a qualifying USD3 deposit, and causes _deployFunds() to supply local waUSDC toward the stale cap. It then borrows the newly supplied liquidity before a keeper can report. For example, USD3 reports 1,000,000 assets, sUSD3 owns shares worth 200,000, and a 50% backing ratio permits 400,000 of deployment. A first borrower consumes 250,000. Materializing a 150,000 markdown lowers USD3's live Morpho claim to 250,000, while the stale cap remains 400,000. A small deposit triggers _deployFunds(), which supplies 150,000 of pre-existing local waUSDC toward the stale cap. The second borrower immediately borrows this newly deployed liquidity. The later report burns approximately 150,000 of sUSD3's value, leaving 50,000 of junior backing and a correct cap of 100,000. However, the full 400,000 remains borrowed and cannot be recalled. If the second 150,000 loan defaults, approximately 100,000 exceeds the remaining junior protection and reaches senior holders. This creates durable unsecured exposure beyond the configured first-loss capacity, reduces available liquidity, and can socialize otherwise unsupported losses across senior USD3 holders.

RECOMMENDATION

Prevent every supply-increasing path while nav() + 2 < totalAssets(). availableDepositLimit() should return zero before assets are transferred, _deployFunds() should not supply during a pending loss, and tend() should remain withdraw-only until report() has recognized the loss and burned the affected junior shares. For complete protection, Morpho should also prevent new borrowing that would leave deployed exposure above the capacity supported by live junior backing. This prevents already-supplied liquidity from being consumed during an unreported-loss interval.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/132/changes/da6bf640bbc7203069cf428e7ce9f1b0095d223d#diff-47476429eaeb244099600dbc5cfb6777c967d823cf785af6a440398d94732089.

MEDIUM M-04 ACKNOWLEDGED

Fee Shares Recapture Their Own Profit.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L776-L819 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 uses Yearn performance-fee shares to implement the configured fraction of yield distributed to sUSD3. Those fee shares subsequently appreciate as the same report's locked profit unlocks, causing sUSD3 and the protocol fee recipient to receive more than the configured fraction. When USD3 reports profit P, Yearn first converts the complete profit into shares at the pre-report price. It mints the configured fraction f of those shares to the fee recipients and locks only the remaining shares in the strategy. The fee shares remain fully outstanding while the strategy-owned locked shares are gradually removed from effective supply. As the locked shares unlock, the fee shares appreciate in the profit that caused them to be minted. For pre-report assets A, profit P, and configured tranche fraction f, the fee shares are initially worth fP, but their value after full unlock is: fP * (1 + P/A) / (1 + fP/A) This is greater than fP whenever 0 < f < 1 and P > 0. If reported profit equals the pre-report assets, a configured 20% allocation ultimately receives approximately 33.33% of the profit. As P/A increases, any nonzero partial allocation approaches the entire reported profit. A factory protocol fee only divides this amplified fee class between recipients; it does not correct the total. The discrepancy is small when interest is reported frequently, but markdown reversals create large one-time recoveries after reported assets have already fallen. For example, beginning with 100 assets, a 90-asset markdown and later 90-asset recovery at a configured 15% tranche share ultimately gives the fee class approximately 57.45 of the recovery rather than 13.5. The surviving senior class receives only approximately 32.55 of the recovered 90. Yearn's accounting is operating as designed, but USD3 describes TRANCHE_SHARE_VARIANT as the yield share distributed to sUSD3 and bypasses Yearn's normal 50% performance-fee limit to support values up to 100%. Using the fee primitive as an exact terminal tranche allocation therefore creates a material semantic mismatch in the scoped USD3 integration.

RECOMMENDATION

Size fee shares according to their terminal value after profit unlock. For existing assets A, supply S, profit P, and desired terminal fraction f, mint aggregate fee shares equal to f * P * S / (A + P - fP), then split those shares between sUSD3 and the protocol recipient and lock the remaining gross-profit shares. Alternatively, implement explicit tranche-allocation accounting that does not let newly issued fee shares participate in their originating profit.

RESOLUTION

Status: Acknowledged.

MEDIUM M-05 RESOLVED

Slash rake is capped by the leftover.

CATEGORY Logical Error LOCATION LCCAuctionLib.sol:265-266 REVIEW Round 1 - Main Review

DESCRIPTION

When a shortfall auction settles, the vault takes a slash fee from the defaulters' margin pool and returns the rest. The fee is `min(marginAwarded * slashFeeBps / BPS, surplus)`, and `surplus` is the part of the pool no bidder took (`LCCAuctionLib.sol:265`).

Capping the fee by the leftover makes it fall to zero at both ends. If a bidder takes nothing, the fee is zero because the award is zero. If a bidder takes the whole pool, the fee is zero because there is no leftover to take it from. The rake shrinks the more a bidder fills.

At the zero-award end the defaulter keeps the entire margin the protocol advertises as forfeited. That end is reachable today: the offer is zero for the whole first step, so anyone who kicks and fills in that window awards nothing and the full pool returns. At the full-fill end the defaulter keeps nothing, and the protocol collects no rake.

The full-fill end is reachable at configurations the validator accepts. `LCCConfigLib.validate` rejects a step decay only above `BPS`, so a decay equal to `BPS` passes. `LCCAuctionLib.offeredPool` then retains nothing from the first step and offers the entire pool, so one fill takes everything and pays no fee.

RECOMMENDATION

Take the fee from the award rather than from the leftover. Charge `marginAwarded * slashFeeBps / BPS` against the awarded margin before it is paid to the bidder, so the rake scales with what is actually taken and cannot be avoided by filling more.

If the fee must stay bounded by the disposed surplus, then bound the award instead: reject a fill that would leave less surplus than the fee it owes. Either way the invariant to enforce is that a larger fill never produces a smaller rake.

Removing the first auction step closes only the zero-award end. A late step offering the whole pool reopens the hole at the other end, since the fee is capped by a surplus that is then zero. Offering the maximum and reserving the maximum fee cannot both hold.

This does not close the separate finding that defaulting can dominate funding. That turns on the penalty being capped by unleveraged margin against a leveraged obligation, which survives any change to how the rake is taken.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/130.

MEDIUM M-06 RESOLVED

Impaired tranche makes LCC default free.

CATEGORY Logical Error LOCATION LCCVault.sol:764-812 REVIEW Round 1 - Main Review

DESCRIPTION

Margin is posted as a bond backing a capital call several times larger, set by `marginRatioBps`. Missing the call forfeits the bond, and that is the whole penalty. For example, at 2x leverage $100 of margin backs a $200 obligation, so walking away costs at most $100.

The issue appears once a borrower default wipes out the junior sUSD3 tranche and cuts into senior `USD3` value. Until a keeper reports it, senior shares still price at the pre-loss value. Forfeited margin is resold to a bidder who covers the missing cash for a cut, but `takeAuction` pays them in those stale-priced shares, so they buy a loss already taken.

That inverts the incentive. Once senior value has fallen about 25% at 2x, or 10% at 5x, walking away costs less than funding. Past roughly 35% at 2x or 14% at 5x, no bidder profits at any price the auction reaches, so nobody bids.

With no bidder there is no award, so the fee taken from it is zero and `_settleAuction` returns the whole margin (`LCCVault.sol:984`). Defaulting is free. Two identical accounts each kept 100% of their margin in one block. Every leveraged account faces the same numbers at once, so the cohort defaults together and the call raises nothing.

RECOMMENDATION

Make the penalty independent of whether anyone bids. Charge the slash fee on the shortfall that went unfilled rather than on the margin actually awarded, so an unfilled auction becomes the defaulter's worst outcome instead of a full refund. Size that penalty against the leveraged obligation the account failed to meet, not against the unleveraged margin, so no level of senior loss lets defaulting dominate funding.

If the rake is kept as designed, close the regime instead. Block auction settlement while a materialized senior loss is still unreported, so bidders are never asked to buy at a knowingly stale share price.

The most practical option is a protocol-owned bidder of last resort funded from the existing insurance fund. It clears the auction whenever no third party will, closing the no-bid regime with capital the protocol already holds.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/130.

MEDIUM M-07 ACKNOWLEDGED

Stale Markdown Lets Early USD3 Holders Exit.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L314-L324 REVIEW Round 1 - Main Review

DESCRIPTION

A USD3 holder can withdraw at a stale share price after a MorphoCredit borrower enters Default but before that borrower's time-grown markdown is materialized, shifting the loss to holders who remain. MorphoCredit derives borrower status from the current timestamp, so an account can enter Default without any transaction updating its markdown. totalMarkdownAmount only changes when the borrower is touched, and the Morpho interface expressly warns that it may be stale. USD3 does not materialize borrower markdowns during withdrawals or report(). report() only accrues market-level interest and does not enumerate borrowers. The withdrawal guard checks whether nav() + 2 < totalAssets(), but nav() uses Morpho's stored aggregate markdown. It therefore still appears healthy while a defaulted borrower's currently calculable markdown remains unmaterialized. A holder can redeem against this stale value and then call the permissionless accruePremiumsForBorrowers() in the same transaction. Morpho materializes the markdown only after the holder has exited. The next USD3 report recognizes the loss against the smaller remaining holder base, and the early holder cannot be charged retroactively. For example, assume USD3 reports 1,000,000 assets, an 800,000 borrower has accumulated 400,000 of unmaterialized markdown, and 200,000 remains liquid. Assuming no remaining junior coverage, an early holder can withdraw 200,000 at the stale price. If the borrower had been touched first, that holder's claim would be approximately 120,000. The approximately 80,000 overpayment is shifted to the holders who remain.

The same stale-markdown window also affects the junior tranche. Before the markdown is materialized and USD3 reports the loss, an sUSD3 holder with a mature cooldown can withdraw underlying USD3 at the stale valuation. Those USD3 shares leave the sUSD3 contract before `_postReportHook` applies the first-loss burn, allowing the exiting holder to avoid its share of the impairment and concentrating the burn on the remaining junior holders. The stale state therefore permits both senior withdrawals at an overstated price and depletion of the junior first-loss pool.

RECOMMENDATION

Before enabling markdown, add a complete borrower registry and market-wide freshness check. If complete synchronization cannot be guaranteed, markdown should remain disabled.

RESOLUTION

Status: Acknowledged.

MEDIUM M-08 PARTIALLY RESOLVED

Exact-floor settlement forges USD3 NAV that extracts fresh depositor liquidity.

CATEGORY Gaming

LOCATION packages/moneymarket-contracts/src/MorphoCredit.sol:_updateMarketMarkdown():L791-L823; packages/moneymarket-contracts/src/MorphoCredit.sol:_applySettlement():L898-L943; packages/moneymarket-contracts/src/MorphoCredit.sol:_requireNewLendi...

REVIEW Round 1 - Main Review

DESCRIPTION

A markdown exactly equal to the assets above the supply-price floor leaves marketInWindDown false. Recapitalization at the depressed Morpho price mints a huge share denominator and raises _minSupplyAssets. When a new loan is drawn and settled, _applySettlement deletes the receivable but clamps totalSupplyAssets to this inflated floor without injecting tokens. USD3.nav values the unbacked claim as real, report preserves it, and deployment targeting treats it as deployed capital.

Impact: The improved production-USD3-path PoC creates more than 40 million USDC of reported but unbacked NAV, then shows a later depositor's real liquidity can be paid to an incumbent while the victim retains an accounted claim that cannot be redeemed. Expected borrower defaults can therefore convert bad debt into withdrawable senior claims and transfer later depositor funds to existing holders.

RECOMMENDATION

Deduct the complete realized settlement loss from totalSupplyAssets and use the share-price floor only as a circuit-breaker, never an asset guarantee. Track the markdown amount actually applied separately from the theoretical borrower markdown, and enter wind-down when the floor is reached before recapitalization.

RESOLUTION

Status: Partially Resolved.

MEDIUM M-09 ACKNOWLEDGED

Tranche fee mint inflates the borrow cap.

CATEGORY Logical Error LOCATION USD3.sol:184-209 REVIEW Round 1 - Main Review

DESCRIPTION

The subordination deploy cap sizes how much of the senior book may sit in the credit market. It reads sUSD3's raw USD3 share balance as the junior first-loss base, converts it at the floor price, and levers it by the inverse of the minimum backing ratio (USD3.sol:191).

`USD3.report` mints the junior its performance-fee share into sUSD3 first, then runs the post-report hook, which tends and re-applies the cap against the now larger balance. On a credit book that profit is accrued interest still owed by the borrower, not cash received. Unpaid interest is therefore booked as first-loss capital and buys borrow headroom at the leverage multiple, in one transaction and with no external capital.

A worked example gives the scale. A junior holding 1 unit of first-loss capital at a 2500 basis point backing ratio starts with a cap of 4 units. Reporting 1.34 units of accrued interest at a 5000 basis point tranche share lifts the base to 1.67 and the cap to 6.68. `USD3._tend` then pushes 1.34 units of fresh collateral into the market. The same report with the fee routed to a treasury leaves the cap flat and recalls 1 unit. The headroom is funded by the loan it extends.

RECOMMENDATION

Base the junior term on capital the tranche has actually received, not on shares minted against unrealized profit. Convert sUSD3's balance through the same locked-profit-aware accounting the strategy uses elsewhere, or exclude fee shares minted in the current report from the cap until that profit has unlocked.

Applying the deploy cap before the fee mint rather than after would also close the same-transaction path, though it leaves the base overstated on later reports.

Add a test that reports pure accrued interest with a nonzero tranche share and asserts the deploy cap does not move.

RESOLUTION

Status: Acknowledged.

MEDIUM M-10 ACKNOWLEDGED

Tranche Changes Reprice Accrued Profit.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L793-L819 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 uses TokenizedStrategy's performance-fee rate as the share of reported profit allocated to sUSD3. syncTrancheShare() replaces that rate by writing the profit-configuration slot directly, but it does not first settle profit accrued under the previous rate. The next report calculates profit from the assets stored by the previous report and applies the single rate active when the new report executes. The replacement rate therefore applies to all profit accumulated since the previous report, including profit earned before the tranche-share change. Increasing the rate transfers historical senior yield to sUSD3, while decreasing it transfers historical junior yield to senior USD3 holders. For crossed profit P, the gross fee allocation changes by P × (newRate - oldRate) / 10,000, subject to rounding and any factory protocol-fee carve-out. Total USD3 assets remain unchanged, but the terminal claims of senior and junior holders change permanently. Mainnet history confirms that this ordering has already reallocated pending profit. In June 2026, a 100%→0% change preceded a $7,863 report, assigning nothing to sUSD3; one week later, a 0%→10% change preceded a $31,307 report, allocating $3,131 under the new rate. This demonstrates that pre-change profit is repriced in production. Users cannot initiate the update or report, but scheduled parameter changes are observable. A user can enter the tranche favored by the replacement rate before the update and receive part of profit earned before their tranche assumed that exposure. Production cooldown requirements delay withdrawal but do not reverse the resulting claim. Reporting immediately before the update is not a complete boundary. Borrower-specific premiums remain outside Morpho's aggregate accounting until the permissionless accruePremiumsForBorrowers() processes a caller-supplied borrower list. Unless preempted, a user can materialize omitted old-period premium after the replacement rate becomes active, causing the next report to allocate it at that rate. Morpho accrues interest under its previous fee before replacing it. USD3's custom slot write performs no equivalent settlement, while a USD3 report cannot settle omitted borrower-local state. The supported rate range is zero to 100%, so the full gross tranche allocation on pending profit can shift between the tranches.

RECOMMENDATION

Settle profit under the old tranche share before storing the new rate. If reporting from the setter is unsafe, checkpoint the accrued profit and rate epoch so the next report applies each rate only to profit earned during its corresponding interval. Execute the checkpoint and rate update atomically.

RESOLUTION

Status: Acknowledged.

MEDIUM M-11 ACKNOWLEDGED

Shutdown exits remain coupled to a failed high-utilization Morpho valuation.

CATEGORY DoS

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:availableWithdrawLimit():L443-L484; packages/moneymarket-contracts/src/usd3/USD3.sol:nav():L684-L685; packages/moneymarket-contracts/src/usd3/sUSD3.sol:availableWithdrawLimit():L284-L300;...

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 evaluates nav() and its Morpho position before checking shutdown, while sUSD3 evaluates USD3.nav() before its own shutdown branch. A full markdown can clamp supply assets to the share-price floor while leaving borrow assets outstanding; AdaptiveCurveIrm is unbounded above 100% utilization, causing both expected and actual interest arithmetic to revert. The production-component PoC reaches this state through the real MarkdownController and standard IRM configuration, then shows the failure after seven days.

Impact: A defaulted credit book can freeze all USD3 and sUSD3 holders out of assets held locally and independently realizable. USD3n can unwrap only into the same frozen USD3. The PoC leaves more than 4,000 local waUSDC plus local junior USD3, yet shutdown maxRedeem and both public redemption paths revert persistently, requiring an upgrade for recovery.

RECOMMENDATION

Handle shutdown before nav, getPosition, and upstream loss checks. During shutdown, value only idle USDC/local waUSDC for USD3 and locally held USD3 for sUSD3, treating failed deployed-position queries as zero. Separately cap AdaptiveCurveIrm above 100% utilization and make expected-balance arithmetic non-bricking.

RESOLUTION

Status: Acknowledged.

MEDIUM M-12 RESOLVED

waUSDC pauses can block willing LCC funders and expose their entire margin to auction.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:availableDepositLimit():L503-L506; packages/moneymarket-contracts/src/usd3/USD3.sol:_deployFunds():L237-L249

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 returns zero deposit capacity whenever waUSDC is paused, so TokenizedStrategy rejects even supply-cap-exempt LCC conduit deposits before `_deployFunds` can use its explicit safe fallback of retaining the received USDC locally. The real-stack PoC starts the pause one second before an LCC funding deadline and shows that a fully funded participant's timely `fundCall` reverts solely at USD3 admission.

Impact: A late wrapper pause can leave no practical time for a separate guardian response. A participant that attempts to satisfy its full call is nevertheless finalized as unfunded and loses its margin; after unpause, the PoC shows an auction filler paying the shortfall and receiving the participant's entire 100 USDC-equivalent margin. Ordinary USD3 deposits and recapitalization are also unnecessarily unavailable during the pause.

The same outcome can arise when Morpho rejects USD3’s synchronous supply because the market is paused or in wind-down, causing the entire LCC funding transaction to revert.”

RECOMMENDATION

Remove the waUSDC pause early return from `availableDepositLimit` while preserving borrower and supply-cap checks. Let `_wrapUSDC` and `_deployFunds` retain deposited USDC idle during the pause, and add direct and exempt-LCC regression tests covering paused admission and deployment after unpause.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/135/changes/39c077a10722f073d81099738a806a3c6791798c.

MEDIUM M-13

PARTIALLY RESOLVED

Backdated zero-to-nonzero DRP creates redeemable phantom USD3 yield.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/MorphoCredit.sol:_accrueBorrowerPremium():L314-L345; packages/moneymarket-contracts/src/MorphoCredit.sol:_snapshotBorrowerPosition():L356-L375; packages/moneymarket-contracts/src/MorphoCredit.sol:_setBo...

REVIEW Round 1 - Main Review

DESCRIPTION

Zero or below-threshold premium accrual returns without advancing lastAccrualTime, while later borrow snapshots replace the paired principal. A prospective zero-to-nonzero DRP update preserves that stale time and latest principal, so the next touch retroactively applies the new rate to principal that did not exist historically and credits the fabricated amount to borrower debt and supplier assets. Unlike historical-premium crystallization, this creates economically nonexistent premium for a pre-existing holder and does not depend on just-in-time share entry.

Impact: An adversarial borrower and related pre-existing USD3 holder can convert fabricated interest into real idle lender liquidity. The PoC fabricates over $1 million on a nearly immediate $10 million draw and lets the holder redeem over $300,000 of phantom yield before settlement, after which remaining USD3/sUSD3 holders bear the reversal and credit loss.

RECOMMENDATION

Atomically update lastAccrualTime whenever principal is snapshotted, including zero and below-threshold cases. On a DRP change, accrue the old rate and unconditionally checkpoint current principal and time before installing the new rate; preserve deferred dust separately.

RESOLUTION

Status: Partially Resolved.

MEDIUM M-14 RESOLVED

Wind-down report rollback lets a cured borrower cash out junior capital and shift its next default to seniors.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:report():L567-L574; packages/moneymarket-contracts/src/usd3/USD3.sol:_postReportHook():L577-L605; packages/moneymarket-contracts/src/usd3/USD3.sol:_applyDeployCap():L344-L360; packages/mon...

REVIEW Round 1 - Main Review

DESCRIPTION

With a binding percentage deployment cap, a floor-crossing markdown reduces USD3's Morpho position by the full loss while the target falls by only that percentage. USD3.report records the loss and tentatively burns sUSD3-held shares, but its unconditional post-report tend then attempts to refill the gap. Morpho rejects new supply in wind-down, rolling back both loss recognition and the junior burn. A borrower with a prepared cooldown can cure its small obligation, reversing the live markdown while most principal remains; after the stale-loss guard clears, it can withdraw junior assets above the still-enforced debt and nominal floors. A later ordinary servicing cycle can default again, at which point the escaped junior capital is unavailable and the residual loss reaches seniors.

Impact: The improved PoC retains a 5% debt-backing ratio, a 75,000-USDC nominal floor, and live 7-day lock/cooldown. The borrower cashes out 200,000 USDC while more than 790,000 debt remains and leaves 700,000 junior USD3. On a newly closed later cycle, the remaining junior balance is exhausted and the senior tranche loses most of its principal. This upgrades the prior report-deadlock finding from withdrawal liveness to direct senior loss.

RECOMMENDATION

Decouple loss accounting and junior burning from fallible rebalancing. During loss reports or Morpho wind-down, skip supply and apply the deployment cap withdraw-only, or separate tending from report so a failed supply cannot roll back recognized losses.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/132.

MEDIUM M-15

ACKNOWLEDGED

Markdown-excluded deployment accounting lets cured borrowers drain reserves and realize losses above MAX_ON_CREDIT.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:_effectiveDeployCapWaUSDC():L205-L210; packages/moneymarket-contracts/src/usd3/USD3.sol:_deployFunds():L237-L273; packages/moneymarket-contracts/src/usd3/USD3.sol:_applyDeployCap():L344-L3...

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 measures deployment using post-markdown supply-share value and omits totalMarkdownAmount even though a markdown neither removes waUSDC from Morpho nor clears borrower debt. At deployment ratio R, markdown M therefore creates roughly M*(1-R) of false headroom, letting a public deposit transfer both its ordinary fraction and pre-existing local reserves into the impaired market. Paying a small obligation restores the markdown on almost all remaining principal; the cured borrower can draw the refill before tend, leaving an illiquid gross position above MAX_ON_CREDIT.

Impact: A strategic borrower can turn an accounting markdown into fresh unsecured liquidity and migrate senior reserves beyond governance's deployment allocation. Once borrowed, tend cannot recall the excess. The extended PoC re-defaults and settles the debt and proves an actual reported senior loss above the 50% deployment target; repeated episodes can expand exposure subject to the broader credit and debt caps.

RECOMMENDATION

Enforce the deployment limit against gross credit exposure by adding USD3's attributable totalMarkdownAmount to both deployed Morpho value and the portfolio denominator, or track gross supplied principal independently. On cure, reserve or recall restored markdown value and prevent new draws that leave USD3 over its target.

RESOLUTION

Status: Acknowledged.

MEDIUM M-16 ACKNOWLEDGED

Attacker-chosen premium crystallization lets new USD3 shares cash out historical unpaid yield.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:_harvestAndReport():L316-L324; packages/moneymarket-contracts/src/Morpho.sol:_accrueInterest():L338-L364; packages/moneymarket-contracts/src/MorphoCredit.sol:accruePremiumsForBorrowers():L...

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 reports and previews accrue aggregate IRM interest but not borrower DRP, penalties, or markdown. An entrant can mint senior shares before permissionlessly checkpointing a long-stale borrower premium; a subsequent routine report allocates the historical receivable across the new shares. Because default markdown is also borrower-touch-driven, routine USD3 reports and withdrawal views can continue valuing the now-defaulted debt at face through profit unlock, allowing the entrant to redeem real USDC before the markdown and loss are recognized. The revised PoC uses an admitted 10% DRP and an economically accurate cycle-ending balance.

Impact: New senior shares can extract a pro-rata portion of DRP economically earned before entry and cash an uncollected receivable out of vault principal. A nonpaying borrower can use a Sybil to externalize the eventual markdown and reported loss onto the remaining USD3 and sUSD3 cohort. Unlike ordinary unreported aggregate interest, the omission is not bounded by report cadence because reports and cycle posting do not advance borrower-specific checkpoints.

RECOMMENDATION

Accrue every active borrower's premium, penalty, and markdown before USD3 issuance pricing, reports, and withdrawals, using an iterable active set or global index. Otherwise halt issuance and redemption whenever borrower checkpoints are stale, and do not report or unlock receivable profit without refreshing default markdown.

RESOLUTION

Status: Acknowledged.

MEDIUM M-17 ACKNOWLEDGED

Permissionless cures let late USD3 depositors snipe senior recovery.

CATEGORY Logical Error

LOCATION packages/moneymarket-contracts/src/MorphoCredit.sol:_trackObligationPayment():L682-L692; packages/moneymarket-contracts/src/MorphoCredit.sol:_afterRepay():L670-L674; packages/moneymarket-contracts/src/MorphoCredit.sol:_updateMarketMarkdo ...

REVIEW Round 1 - Main Review

DESCRIPTION

A permissionless full-obligation repayment clears a borrower's status and immediately reverses its entire markdown in MorphoCredit, restoring supplier NAV while most debt may remain. USD3 deposits still price against impaired stored totalAssets until a keeper report. An attacker can atomically order Helper.repay for the borrower followed by its own USD3 deposit, entering after recovery is realized but before it is reflected in the share price.

Impact: The revised zero-fee PoC mints roughly twice the live-NAV fair shares: a 500 USDC stale-price deposit becomes about 751 USDC and is redeemed after the normal unlock, extracting over 250 USDC of senior recovery while the original senior remains below principal and almost all borrower debt remains. With greater cap headroom, the bundler can capture most restored markdown; keeper discipline cannot interpose inside the repay-plus-deposit bundle.

RECOMMENDATION

Accrue/reconcile live NAV atomically before deposit share conversion, or block deposits whenever nav materially exceeds stored totalAssets. Ensure a permissionless cure cannot create a post-realization stale entry price.

RESOLUTION

Status: Acknowledged.

MEDIUM M-18 ACKNOWLEDGED

Junior holders can withdraw fees on unpaid income before default.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/MorphoCredit.sol:_updatePositionWithPremium():L275-L301; packages/moneymarket-contracts/src/usd3/USD3.sol:_harvestAndReport():L316-L324; packages/moneymarket-contracts/lib/tokenized-strategy/src/Tokeniz...

REVIEW Round 1 - Main Review

DESCRIPTION

MorphoCredit capitalizes borrower premium into supplier assets without collecting tokens, and USD3 reports that receivable growth as profit. TokenizedStrategy immediately mints performance-fee USD3 shares to sUSD3. sUSD3 then reports those shares as its own profit and, after ordinary unlocking and cooldown, permits holders to withdraw any value above the live debt-backing floor. While the borrower is Current, neither unrealized-loss guard triggers, so fee shares backed by still-unpaid debt can leave sUSD3. If a later scheduled obligation defaults, the guards then close and a markdown lowers NAV, but USD3's loss hook can burn only locked USD3 shares and shares still held at the sUSD3 address. Previously withdrawn fee shares are outside that burn and survive as pari-passu claims on real liquidity and recoveries. The passing PoC uses zero Yearn protocol fee, eighteen nonempty 2% monthly obligations paid in full, then a nineteenth nonempty obligation that genuinely ages through grace, delinquency, and markdown before settlement.

Impact: A junior holder can convert fees assessed on an ultimately uncollected receivable into ordinary USD3 before the credit loss exists on-chain. Those shares survive the later loss and divert real post-default liquidity from the remaining lender pool. In the PoC, a holder who contributed $500k of genuine junior principal escapes more than $1m of fee-derived USD3 and redeems over $1m USDC after a genuine default and write-off; the live junior balance, rather than the escaped shares, absorbs the reversal. Repeated high-premium credits can scale this transfer with accrued income and pool liquidity, deplete durable first-loss capital, and dilute senior recovery even though operators service every prior cycle correctly.

Uncollected interest also raises USD3’s reported value directly, allowing senior holders to withdraw paper profit before a later default shifts the loss to remaining suppliers.

RECOMMENDATION

Do not make fees on uncollected credit receivables freely withdrawable. Charge performance fees only on realized cash collections or maintain a per-receivable high-water mark and clawback reserve. At minimum, escrow/lock fee-minted USD3 inside a burnable tranche bucket until the associated borrower income is collected, exclude that value from sUSD3 withdrawable excess and subordination backing, and ensure later markdowns/defaults reverse any previously accrued uncollected fee before holders can transfer or redeem it.

RESOLUTION

Status: Acknowledged.

LOW L-01 ACKNOWLEDGED

Temporary Deposits Bypass MAX_ON_CREDIT.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L235-L273 REVIEW Round 1 - Main Review

DESCRIPTION

An approved borrower can use a temporary USD3 deposit to create additional borrowable liquidity, withdraw the temporary capital from the shared local buffer, and leave USD3 permanently deployed above `MAX_ON_CREDIT`.

`availableDepositLimit()` rejects a receiver only when that receiver already has borrow shares. A borrower with no current debt can therefore deposit before borrowing, or deposit to a clean controlled receiver. During the deposit, `_deployFunds()` includes the temporary assets in total waUSDC and immediately supplies enough liquidity to MorphoCredit to reach the enlarged `MAX_ON_CREDIT` target.

The borrower can then borrow the newly deployed liquidity. Because ordinary USD3 has no withdrawal delay, the clean receiver can immediately redeem its temporary shares. `_freeFunds()` consumes local waUSDC first, allowing the temporary capital to exit without recalling the liquidity just deployed to Morpho. No post-withdraw hook verifies that deployment remains below `MAX_ON_CREDIT`. Once the borrower has consumed Morpho's cash, a later `tend()` cannot restore the ratio because the excess deployment cannot be withdrawn until the borrower repays.

For example, with an 85% deployment ratio and `H` honest assets, USD3 initially holds `0.85H` in Morpho and `0.15H` locally. The borrower can borrow approximately `0.1765H`, deposit it through a clean receiver, cause the remaining `0.15H` buffer to be deployed, redeem the temporary `0.1765H` from the resulting local balance, and then borrow the remaining Morpho liquidity. USD3 finishes with approximately `H` of unsecured debt and no local buffer despite the intended 85% limit.

Production configuration note: At the time of review, the deployed configuration has MAX_ON_CREDIT = 10000 and USD3 is at or above its 75m supply cap, so this path currently has no exploitable deposit headroom. Both values are mutable protocol parameters, and lower deployment ratios are documented and covered by committed tests as supported operating configurations. Reducing MAX_ON_CREDIT below 100% while restoring deposit headroom activates the issue without a contract upgrade.

RECOMMENDATION

Do not allow reversible deposits to create immediately borrowable deployment capacity. Enforce the deployment cap again during withdrawals and revert or limit withdrawals whenever the resulting asset base would leave deployed assets above the effective cap. Alternatively, require newly deposited capital to remain locked until any deployment attributable to it can be recalled.

RESOLUTION

Status: Acknowledged.

LOW L-02 ACKNOWLEDGED

Beacon upgrade bricks vault allowances.

CATEGORY Upgradeability LOCATION LCCVault.sol:150-175 REVIEW Round 1 - Main Review

DESCRIPTION

A routine implementation upgrade can permanently block funding in every deployed vault. Funded USDC is wrapped twice: first into `USD3`, then into the NotificationVault that issues `USD3n`. Both addresses are implementation immutables shared by every proxy behind the beacon, not per-vault storage. A vault approves them only in `initialize` (`LCCVault.sol:174-175`), which is `initializer`-guarded with no reinitializer and no re-approve entrypoint.

Point the beacon at an implementation naming a different NotificationVault and every live proxy holds a zero allowance toward it. `fundCall` reverts inside that deposit for every funder of every facility, and no on-chain call can restore the approval. The deploy script hardcodes USD3 but reads the `NotificationVault` from the environment, requiring only that its `asset()` match.

No malice is needed: shipping a fixed NotificationVault is ordinary maintenance. The upgrade checklist covers storage layout and library re-linking but never allowance migration. A cohort inside a funding window then misses the all-or-nothing deadline and forfeits its posted margin. Rolling the beacon back restores future funding but cannot reverse a finalized slash.

RECOMMENDATION

Add a `refreshApprovals` entrypoint that re-grants USDC to USD3 and USD3 to the NotificationVault from the current immutables. Have the delivery helper top up an insufficient allowance before it deposits, so a stale approval degrades into an extra approve rather than a reverted funding call.

Better still, write both addresses into per-vault storage during `initialize`, so swapping the implementation cannot change the delivery route under a live proxy. At minimum, add allowance migration to the upgrade checklist. The wiring check should also assert a nonzero allowance toward both wrappers, so it fails loudly instead of a funding window failing silently.

RESOLUTION

Status: Acknowledged.

LOW L-03 ACKNOWLEDGED

Blocked live exiter has no funding path.

CATEGORY DoS LOCATION LCCVault.sol:764-812 REVIEW Round 1 - Main Review

DESCRIPTION

A margin-token outage halts nearly every operation here: deposits, exit and shutdown claims, treasury sweeps, amortized funding and margin-awarding fills all revert. Rolling moves no margin and is the only surviving way to meet an obligation, but a live exiter cannot roll (`LCCVault.sol:776`). The only group barred from the sole working action is the one that loses everything by missing its deadline. Non-exiting accounts ride it out; slashing moves no margin, so the clock never stops.

Amortizing pushes released margin to the account with no destination argument (`LCCVault.sol:808`), so it reverts. Third-party funding hardwires rolling off, no function cancels an exit request, and claiming is itself a margin push. Forfeiture is total; the return pool restores the margin as active, not claimable.

Nothing detects the condition. The funding side checks a paused wrapper at six sites, since a pause blocks every transfer; the margin side, where the penalty is the whole bond, has none. The trigger is a margin token refusing transfers during the funding window, by pause or address restriction. Config accepts any nonzero asset, and the intended yield-bearing stablecoins restrict routinely.

RECOMMENDATION

Give the release a destination. Add a receiver parameter to the self-funding path, or accrue the released margin to a per-account claimable balance that a separate call delivers. The codebase already contains that pull form: treasury disposal accrues to `pendingTreasuryMargin` and a permissionless sweep delivers it, precisely so the payout cannot be bricked by a recipient refusing transfers. A blocked account then pays one extra claim instead of its whole position.

Allowing a live exit request to be withdrawn is a cheap, independent second fix, since the account could then roll.

Pause awareness on the margin side helps too, but as a supplement rather than a substitute. `USD3` can check `Pausable(WAUSDC).paused()` because waUSDC is a hardcoded constant with a known interface; `marginAsset` is configurable per vault, `paused()` is not part of ERC20, and a token-wide check would miss the likelier per-address restriction.

RESOLUTION

Status: Acknowledged.

LOW L-04 ACKNOWLEDGED

sUSD3 entry misses USD3 loss guard.

CATEGORY Validation LOCATION USD3.sol:684; sUSD3.sol:250-293 REVIEW Round 1 - Main Review

DESCRIPTION

The junior vault closes for exits whenever USD3 carries an unrecognised loss. `sUSD3.availableWithdrawLimit` compares `USD3.nav` against USD3's reported total assets and returns zero when nav is lower.

`sUSD3.availableDepositLimit` consults no such signal (sUSD3.sol:250). On the identical on-chain state the first-loss tranche is shut for leaving and open for joining, so a newcomer joins the burn base for a loss that predates them.

A recent fix mirrored the tranche-level staleness check from the exit path onto the entry path at sUSD3.sol:254, closing the window between the first-loss burn and the junior vault's own report. The USD3-level guard was not mirrored. It is the earlier of the two signals: nav falls the moment a borrower is settled, while the balance the tranche-level check reads only falls once the burn lands.

Settlement is operator-initiated, so there is no attacker and no profit motive. The protection rests on the operator bundling the report with the settlement rather than on code.

RECOMMENDATION

Mirror the second exit guard onto the entry path. Return zero from the junior deposit limit while USD3's nav plus two is below USD3's reported total assets, the same condition the withdraw limit already applies.

The tranche-level check added earlier is not a substitute. It reads the USD3 balance held by the junior vault, and that balance only falls once the loss is burned, which is strictly later than the moment the loss becomes visible in nav.

Add a regression test asserting both limits are zero across the whole pre-burn window, so a future change to one hook has to be made to the other.

RESOLUTION

Status: Acknowledged.

LOW L-05 RESOLVED

Unstake reverts on a paused wrapper.

CATEGORY DoS LOCATION USD3.sol:613-615 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 treats a paused Aave wrapper as a condition to tolerate, not to revert on. The deploy, tend, tend-trigger and available-withdraw paths all short-circuit while the wrapper is paused, because a paused wrapper blocks every waUSDC transfer.

`USD3._postTransferHook` carries no such guard. On any share transfer out of the junior vault it re-applies the deployment cap via `USD3._applyDeployCap` (USD3.sol:614), which calls `USD3._withdrawFromMorpho` whenever the deployed balance exceeds the target. That withdrawal moves waUSDC and reverts under the pause, and the revert propagates out of the share transfer and out of the junior holder's redemption.

Every junior exit is then blocked for the length of an outage the contract elsewhere absorbs deliberately. The withdrawal branch is reached only when the deployment cap binds. At the live configuration it does not, since the backing ratio is zero and the credit ratio is at maximum. It arms as soon as either is moved, including the documented emergency stop that sets the credit ratio to zero.

RECOMMENDATION

Add the guard the sibling call sites already carry. Skip the cap re-application in the post-transfer hook while the wrapper is paused, so a junior unstake degrades to a deferred rebalance rather than a revert.

More broadly, do not embed an unbounded external market call inside an ERC20 transfer. Make the cap re-application failure tolerant, and persist a pending-rebalance flag that the keeper trigger reports once the pause lifts, so no market-side revert can block junior withdrawals and no deferred recall is silently forgotten.

Add a regression test that pauses the wrapper with the backing ratio set nonzero and asserts a junior redemption still succeeds.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/132.

LOW L-06 RESOLVED

Revoked Bypass Reuses Old Cooldown.

CATEGORY Logical Error LOCATION src/usd3/NotificationVault.sol:L97-L185 REVIEW Round 1 - Main Review

DESCRIPTION

NotificationVault allows management to exempt trusted custody or market accounts from its withdrawal cooldown. Revoking the exemption changes only cooldownBypass[account] and preserves any existing cooldown ticket. While bypassed, an account can start a cooldown and transfer away every share without reducing its recorded cooldown quota because _preTransferHook() returns early. After management revokes the bypass, the old ticket becomes active again. If the account receives fresh USD3n during the retained withdrawal window, availableWithdrawLimit() applies the old quota and timestamps to that new balance. The account can therefore redeem shares that never served the cooldown while the original seasoned shares remain outstanding elsewhere. The committed invariant handler avoids this state by making the account cancel an orphaned cooldown before revocation. Production setCooldownBypass() performs no equivalent cleanup. No shares are duplicated, and the early withdrawal is bounded by the stale quota. However, the revoked account can unwrap newly received collateral without serving the configured notification period.

RECOMMENDATION

Clear an account's cooldown atomically whenever its bypass status changes. In particular, setCooldownBypass(account, false) should delete any existing ticket before re-enabling cooldown enforcement. Also reject startCooldown() while the account is bypassed, preventing bypassed transfers from leaving reusable cooldown state.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/134.

LOW L-07 ACKNOWLEDGED

Transferred Shares Bypass Minimum Deposit.

CATEGORY Validation LOCATION src/usd3/USD3.sol:L546-L550 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 enforces `minDeposit` only when the receiver's current USD3 share balance is zero. Since USD3 shares are freely transferable, the balance does not prove that the receiver ever completed a qualifying deposit.

An existing holder can transfer one raw USD3 share to a fresh address. That address is immediately treated as an established depositor and can call `deposit()` or `mint()` with amounts below `minDeposit`; the same call from an otherwise identical zero-balance receiver reverts.

The seed is reusable. The receiver can deposit one asset base unit, redeem the newly minted share, retain the transferred seed, and repeat indefinitely. This permits arbitrary dust positions despite the configured admission floor and lets a seeded account repeatedly invoke deposit-time processing of the strategy's full loose balance using a sub-minimum contribution.

The minimum was previously hardened against the max-value deposit bypass, but using a transferable token balance as the first-deposit marker leaves the same admission rule avoidable through share seeding.

RECOMMENDATION

Do not infer depositor initialization from a transferable share balance. Track whether an address completed a qualifying deposit, enforce minDeposit on every non-exempt deposit or mint, or require the receiver's post-transaction position to meet the minimum without allowing transferred dust to satisfy it.

RESOLUTION

Status: Acknowledged.

LOW L-08 ACKNOWLEDGED

Slash Sync Defeats Auction Off-Switch.

CATEGORY Logical Error LOCATION src/lcc/LCCVault.sol:L282-L290 REVIEW Round 1 - Main Review

DESCRIPTION

`setMaxAuctionAwardBps()` uses the `synced` modifier. Once an unfunded call reaches its funding deadline, the modifier finalizes the slash before the setter body executes. Finalization reads the old nonzero award cap and creates a live auction. The setter then detects the live auction and reverts, rolling back both finalization and the attempted cap reduction.

The owner therefore cannot set the award cap to zero after the shortfall becomes final but before the auction is created. A filler can instead call `finalizeEpochSlash()` and `takeAuction()` atomically, creating and filling the auction under the old cap with no transaction boundary in which governance can disable it.

In the isolated ordinary-scale sequence, one account leaves a 50-unit shortfall backed by 50 units of slashed margin. At the first nonzero auction step, the filler supplies the shortfall and receives 25 margin units. A pre-deadline cap update prevents the auction, but the identical post-deadline update cannot take effect until after the fill and settlement are irreversible.

The documented runtime auction off-switch is therefore unavailable precisely when the shortfall becomes actionable.

RECOMMENDATION

Add a strict-reduction path that records zero or a lower award cap before slash synchronization when no fill has occurred. Finalization should consume the pending reduced cap, or the owner should be able to cancel an unfilled auction without permanent shutdown. Preserve the existing no-repricing rule after the first fill.

RESOLUTION

Status: Acknowledged.

LOW L-09 RESOLVED

Zero Unlock Bypasses USD3 Loss Buffer.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L567-L604 REVIEW Round 1 - Main Review

DESCRIPTION

When USD3 has locked profit and a materialized but unreported loss, management can release the profit buffer before recognizing the loss, causing sUSD3 to absorb value that the buffer would otherwise cover. USD3.report() records supply and assets before TokenizedStrategy's report. On a loss, TokenizedStrategy burns remaining locked-profit shares. USD3 subtracts that burn from the shares charged to sUSD3, so locked profit absorbs the loss first and sUSD3 covers only the remainder. The inherited setProfitMaxUnlockTime(0) path breaks this ordering. It immediately burns every strategy-owned locked share without checking live NAV or updating stored totalAssets. If called before report(), the released profit increases the value of existing USD3 shares. The later report finds no locked shares to burn, so USD3._postReportHook() burns sUSD3-owned shares instead. For example, suppose USD3 has 1,100 stored assets and shares, including 100 locked, 100 junior-owned, and 900 senior-owned shares. A materialized 50-asset loss reduces live NAV to 1,050. Reporting first burns 50 locked shares, leaving senior and junior positions worth approximately 945 and 105 after the remaining unlock. Zeroing first burns all locked shares and raises the stored share price to 1.10; reporting then burns approximately 45.45 sUSD3-owned shares, leaving the positions worth approximately 990 and 60. The same assets remain, but approximately 45 move from junior to senior solely because the setter ran before the loss checkpoint. USD3 management is a 24-hour TimelockController, so zero-unlock operations are observable. A senior holder can hold USD3 and materialize already-eligible borrower markdowns during the delay or front-run execution. An atomic zero-before-report batch still releases locked profit before charging the materialized loss to sUSD3. After report, the holder retains the increased redeemable claim. While junior shares suffice, the transferred value is roughly the senior fraction of non-locked shares multiplied by the lesser of the pending loss and locked-profit buffer. This can be material after a large profit report, including markdown recovery, and irreversibly shifts value from sUSD3's first-loss capital to senior holders.

RECOMMENDATION

Before allowing USD3's profit unlock time to be set to zero, recognize any pending profit or loss under the existing lock state. USD3 can directly intercept setProfitMaxUnlockTime(uint256) and revert while nav() + tolerance is below totalAssets(), requiring report() to recognize the loss first. Alternatively, checkpoint USD3 accounting atomically inside every zero-unlock transition before forwarding the configuration change to TokenizedStrategy.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/134.

LOW L-10 RESOLVED

Premium accrual can brick a borrower.

CATEGORY Math LOCATION MathLib.sol:64-74 REVIEW Round 1 - Main Review

DESCRIPTION

`MathLib.wInverseTaylorCompounded` recovers a per-second rate from a growth factor using a 3-term alternating series. Solidity evaluates it left to right under checked arithmetic, so the subtraction runs before the third term is added (MathLib.sol:71). Once the growth factor exceeds three, the second term is more than twice the first and the expression underflows and panics. The docstring states no bound.

`MorphoCredit._accrueBorrowerPremium` reaches it by dividing current debt by the balance recorded at the last accrual. When it panics, that borrower can no longer borrow, repay or be settled, and the permissionless accrual entrypoint reverts. Only a proxy upgrade restores the account.

Reachability is remote as deployed. Between accruals only market-wide base interest moves the numerator, and the measured live base rate is about 0.72 percent a year, so tripling takes roughly 150 years. Simulated over 20 years, the account never bricks.

The nearer trigger is the penalty path, which uses the operator-supplied ending balance from the repayment obligation. That figure is never checked against real debt, and a value below a third of it panics on the first penalty accrual.

RECOMMENDATION

Bound the series to its safe numerical domain. Replace the approximation or provide a mathematically sound, non-reverting fallback for growth factors outside that domain. A rejecting guard alone is insufficient because it preserves the borrower-servicing denial of service.

Correct the docstring at the same time. It warns that inputs above `2.5e18` are not recommended, but states no enforced upper bound or panic threshold, which makes the missing guard easy to miss at the call sites.

Derive `endingBalance` from, or verify it against, an authoritative cycle-end debt snapshot. If no authoritative value is available, enforce a reasonable consistency bound and provide a safe correction path before the value is used as a divisor.

This failure is independent of the protocol-fee configuration: the panic is in the base-growth term and arises with the fee at zero.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/135/changes/d0d7bb37b2e5f0737a2f9294997af04572923912.

LOW L-11 ACKNOWLEDGED

Fee changes retroactively reallocate realized borrower premiums.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/Morpho.sol:setFee():L124-L136; packages/moneymarket-contracts/src/MorphoCredit.sol:accruePremiumsForBorrowers():L146-L153; packages/moneymarket-contracts/src/MorphoCredit.sol:_updatePositionWithPremium(...

REVIEW Round 1 - Main Review

DESCRIPTION

setFee checkpoints aggregate IRM interest but leaves borrower premium timestamps untouched before replacing market.fee. The next borrower touch calculates the full old interval yet charges its premium at the new fee, making transaction ordering assign up to 365 days of uncheckpointed DRP or penalty wholly to either side of a fee transition. An updated production PoC shows the misallocation remains after the borrower fully repays before fee withdrawal, so it is distinct from the prior paper-fee withdrawal-priority issue.

Impact: If market fees are used, a fee transition can reallocate up to the new fee fraction of a large portfolio-wide premium backlog between suppliers and the fee recipient based solely on call ordering. Under maximum permitted parameters, the shifted realized revenue can be economically material.

RECOMMENDATION

Store timestamped fee epochs and split lazy premium accrual across fee rates active during each elapsed interval, or atomically checkpoint every live borrower under the old fee before changing it.

RESOLUTION

Status: Acknowledged.

LOW L-12 ACKNOWLEDGED

High USD3 PPS magnifies first-loss floor rounding into material senior losses.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:_postReportHook():L577-L604; packages/moneymarket-contracts/src/usd3/USD3.sol:_subordinationDeployCapWaUSDC():L184-L202; packages/moneymarket-contracts/lib/tokenized-strategy/src/Tokenized...

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 floor-rounds the junior shares required to absorb a reported loss. Profit unlocking after ordinary redemptions can leave a very high USD3 price per indivisible base share because fully unlocked strategy-held shares are excluded from effective total supply. The deployment cap counts an sUSD3-held base share at its full asset value, yet a subsequent loss smaller than that value converts to zero burn shares.

Impact: A default economically covered by the junior tranche can materially reduce senior PPS while leaving the junior share intact. The passing PoC creates an approximately $1 million USD3 base-share value, deploys and writes off slightly less than that amount, burns zero junior shares, and shows seniors bearing over 98% of the loss while the junior exits with over 98% of its pre-loss value.

RECOMMENDATION

Round the required junior burn upward or carry the unabsorbed asset residual across reports, and ensure value admitted by the subordination deployment cap remains slashable. Alternatively enforce a PPS/effective-supply granularity bound. Add a cross-multiplied invariant that losses no greater than pre-report junior value cannot reduce senior PPS.

RESOLUTION

Status: Acknowledged.

LOW L-13 ACKNOWLEDGED

Unlimited DEBT_CAP overflows waUSDC conversion and blocks sUSD3 deposits.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/sUSD3.sol:getSubordinatedDebtCapInUSDC():L454-L478; packages/moneymarket-contracts/src/usd3/sUSD3.sol:availableDepositLimit():L250-L278; packages/onchain-ethereum-type4/src/__onchain/statatokenv2-4...

REVIEW Round 1 - Main Review

DESCRIPTION

DEBT_CAP accepts type(uint256).max and MorphoCredit's ceiling check naturally uses it as an unlimited cap, consistent with the codebase's own unlimited-cap tests. sUSD3 instead passes that value to waUSDC.convertToAssets. Because the live waUSDC exchange rate exceeds one, the conversion overflows, causing the subordinated-cap view and all deposit/mint limit paths to revert.

Impact: A supported unlimited-debt configuration disables all new sUSD3 deposits. This prevents replenishing junior first-loss backing, potentially leaving credit deployment unavailable after junior depletion until governance restores a finite cap.

RECOMMENDATION

Special-case type(uint256).max before conversion and return a saturated capacity. Use mulDiv or explicit saturation for the subsequent ratio computation and test the maximum value at exchange rates above one.

RESOLUTION

Status: Acknowledged.

LOW L-14 ACKNOWLEDGED

Share-floor phantom liquidity makes USD3 loss reports revert and bricks deposits.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:getMarketLiquidity():L151-L158; packages/moneymarket-contracts/src/usd3/USD3.sol:_applyDeployCap():L344-L360; packages/moneymarket-contracts/src/usd3/USD3.sol:_withdrawFromMorpho():L405-L4...

REVIEW Round 1 - Main Review

DESCRIPTION

A wind-down settlement preserves minimum Morpho supply assets purely as a share-price floor even when Morpho holds no loan tokens. USD3 derives withdrawable market liquidity as totalSupplyAssets minus totalBorrowAssets, treating those unbacked protected assets as cash. With a binding deployment cap, report burns junior shares and lowers the deployment cap, then its mandatory post-report rebalance attempts to withdraw the phantom balance. The token transfer fails and rolls back the entire report, including loss recognition and junior burning.

Impact: After a full-book default, USD3 cannot complete junior-first loss reporting or update recorded assets. The vault remains in stale accounting until irreversible strategy shutdown or an upgrade.

RECOMMENDATION

Bound Morpho withdrawable liquidity by the actual loan-token balance and wrapper redeemability rather than only accounting supply-minus-borrow. Make post-report rebalancing best-effort so unavailable recall cannot roll back loss recognition.

RESOLUTION

Status: Acknowledged.

LOW L-15 RESOLVED

USD3 maxDeposit and maxMint advertise cap fragments rejected by minDeposit.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:availableDepositLimit():L503-L526; packages/moneymarket-contracts/src/usd3/USD3.sol:_preDepositHook():L533-L551; packages/moneymarket-contracts/src/usd3/base/BaseHooksUpgradeable.sol:depos...

REVIEW Round 1 - Main Review

DESCRIPTION

availableDepositLimit returns raw supply-cap headroom without applying the first-deposit minimum. When positive headroom is below minDeposit, maxDeposit and the derived maxMint advertise nonzero limits that deposit and mint reject for a non-exempt zero-balance receiver. This is distinct from the accepted transferred-share bypass: no transfer occurs, and fixing either omission alone leaves the other intact.

Impact: ERC-4626 integrations that submit the advertised maximum deterministically revert for new receivers while the cap fragment persists. Existing holders can consume the fragment, but no principal is lost and configuration or withdrawals can restore onboarding, limiting severity to Low.

RECOMMENDATION

Return zero from availableDepositLimit when a non-exempt zero-balance receiver faces cap headroom below minDeposit, and test that every nonzero advertised deposit/mint maximum succeeds if state is unchanged.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/136/changes/f3ec842978911a0f970d76c771268a02680e250a.

LOW L-16 ACKNOWLEDGED

One defaulted borrower freezes exits.

CATEGORY DoS LOCATION USD3.sol:443-458 REVIEW Round 1 - Main Review

DESCRIPTION

One defaulted borrower can freeze withdrawals for every holder. `availableWithdrawLimit` returns zero for everyone whenever `nav()`, the live value of the vault's lending position, falls more than 2 asset units below the booked `totalAssets` (USD3.sol:456). A markdown, the markdown applied to a defaulted loan, lowers `nav()` and trips it. The junior tranche applies the same condition, so both stop.

The trigger is absolute, not proportional. Right after a keeper `report()` the two are equal, so a markdown of 3 units, 0.000003 USDC, halts the pool. Markdown is opt-in per borrower; once an owner enables it anyone can trigger it through the unauthenticated `accruePremiumsForBorrowers`. Unrelated holders freeze too, and only a keeper or management can lift it.

The in-code comment calls the halt transient because accruing interest lifts `nav()` back. That holds for rounding drift, not for markdown, which grows by a 730th of the defaulted loan per day (MarkdownController.sol:110). The interest cushion is about $13.4k and regrows near $8.2k per day, so a position above roughly $6.0M re-freezes faster than keepers clear it: 8% of the book. No borrower is marked down today.

RECOMMENDATION

Stop conditioning redemption liveness on a markdown that has not yet been reported. Either price exits directly off `nav()` in `availableWithdrawLimit` instead of halting whenever it trails `totalAssets`, or make loss realization permissionless so any holder can re-base `totalAssets` without waiting for a keeper.

If the halt is kept, express it as a materiality threshold in basis points of `totalAssets` rather than the fixed two-unit tolerance, so a dust markdown cannot freeze the whole vault. Before enabling markdown for a borrower, compare the position against the size at which markdown accrual overtakes interest regrowth.

Fix the asymmetry, not one side of it: while `nav()` trails `totalAssets`, deposits still mint at the stale price with no guard. Entry and exit should share one definition. A separate finding asks for this same condition to be mirrored onto junior entry, so settle the form once and apply it to both sides rather than relaxing exits while entry stays unguarded.

RESOLUTION

Status: Acknowledged.

LOW L-17 RESOLVED

Funding depends on an unbound USD3 flag.

CATEGORY Validation LOCATION LCCVault.sol:815-820 REVIEW Round 1 - Main Review

DESCRIPTION

`LCCVault._deliverWrapped` funds a call by depositing USDC into USD3 with the vault as receiver (`LCCVault.sol:817`). A non-exempt receiver can deposit only within the general USD3 supply-cap headroom. If a call obligation exceeds the remaining headroom, the deposit reverts; setting `supplyCapExempt[vault]` is therefore an operational precondition for reliable funding.

Neither the vault initializer nor the factory registry checks that flag, so a registered vault can exist without it and USD3 management can clear it later. The canonical creation script tries to prevent this by batching vault creation and both USD3 registry writes atomically.

That batch no longer matches the deployed authority split. The planned factory owner is the Safe, while live USD3 management is the 24-hour timelock. `CreateLCCVaultSafe` requires both authorities to equal the Safe, so it cannot execute as written against the current roles. Operators must instead pre-authorize the predicted vault address or coordinate separate governance transactions, and no on-chain invariant enforces the required ordering.

Current supply-cap headroom is positive and mutable, so an unexempt vault is not necessarily blocked immediately. The failure becomes live once shared headroom is exhausted or a call exceeds what remains. `fundCall` then reverts for otherwise solvent funders; unless governance fixes the flag or pauses the facility clock before the deadline, those accounts become slash-eligible. This is an unenforced deployment and governance precondition, not evidence that every call is presently blocked.

RECOMMENDATION

Bind registration and exemption on-chain. Require the USD3 supply-cap exemption before the factory records a vault, and prevent the exemption from being cleared while that vault remains registered.

Update the deployment flow for the current authority split. The timelock can grant the exemption and ring-fence permission to the factory's predicted CREATE2 address before deployment; the Safe can then create the vault, and the script should verify both flags afterward. Remove the incorrect requirement that USD3 management equal the Safe and instead verify the expected timelock authority.

Until the invariant is enforced, treat an exemption-related funding failure as a facility incident: pause the effective clock or otherwise make the affected funding window non-slashable while governance repairs the flag.

RESOLUTION

Status: Resolved.

LOW L-17 CONTINUED 2/2 RESOLVED

Funding depends on an unbound USD3 flag.

RESOLUTION

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/136/changes/1b19e1a2ce48286e49fe2a39742561840ff2c7a8.

LOW L-18 ACKNOWLEDGED

Consumed junior backing remains borrowable when USD3 cannot rebalance.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:_subordinationDeployCapWaUSDC():L184-L203; packages/moneymarket-contracts/src/usd3/USD3.sol:_tend():L328-L338; packages/moneymarket-contracts/src/usd3/USD3.sol:_postReportHook():L577-L605; ...

REVIEW Round 1 - Main Review

DESCRIPTION

A loss report can burn all USD3 shares held by sUSD3 and reduce the subordination deployment cap to zero. Although `_postReportHook` calls `_tend` after the burn, `_tend` returns without recalling excess Morpho supply whenever USD3 is shutdown or waUSDC is paused, and no persistent recall-required state is recorded. MorphoCredit's borrow hook does not check USD3 shutdown or whether live junior backing still supports supplied liquidity. An already-underwritten current borrower can therefore draw all residual liquidity before emergency withdrawal or a later successful tend.

Impact: A junior-covered default can cascade into loss of essentially all remaining deployed senior capital during an incident shutdown or waUSDC recovery window. The passing PoC has a second borrower drain 8m of unsupported liquidity after a 2m default consumes all junior backing; the second default reduces senior USD3 PPS by roughly 80% and forces wind-down.

Shutdown is another trigger for the same missing borrow-admission check. A markdown cure can clear sUSD3's pending-loss gate, permitting the borrower to withdraw junior backing during shutdown; because `MorphoCredit._beforeBorrow` checks neither strategy shutdown nor live junior capacity, the borrower can then redraw before any recall.

RECOMMENDATION

Do not commit a backing-reducing junior burn while unsupported liquidity remains borrowable without recording an enforceable stop. Persist an over-subordination-cap or recall-required state and reject new MorphoCredit borrowing until recall succeeds. Couple USD3 shutdown to a borrowing stop, and handle the waUSDC-paused report path atomically or retain the stop until the cap is restored.

RESOLUTION

Status: Acknowledged.

LOW L-19 ACKNOWLEDGED

One-share sUSD3 inflation steals up to 25% from the next junior depositor.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/sUSD3.sol:_harvestAndReport():L121-L125; packages/moneymarket-contracts/src/usd3/sUSD3.sol:availableDepositLimit():L247-L278; packages/moneymarket-contracts/lib/tokenized-strategy/src/TokenizedStra...

REVIEW Round 1 - Main Review

DESCRIPTION

sUSD3 has neither virtual/dead shares nor a meaningful first-deposit minimum. Its harvest hook reports the entire USD3 balance, including direct transfers, while TokenizedStrategy floor-rounds assets * effectiveSupply / totalAssets. An attacker initializes sUSD3 with one base-unit share, transfers D USD3 directly to it, waits for a routine fee-free keeper report, and waits until the locking shares unlock and are excluded from effective supply. Its one share then represents C=D+1 assets. A victim deposit V with C <= V < 2C receives only one share, after which the attacker withdraws half of C+V. Net of recovering C, attacker profit is (V-C)/2 and approaches V/4. The PoC demonstrates approximately 500,000 USD3 stolen from a 1,999,999 USD3 victim deposit after one report/unlock.

Impact: An unprivileged first junior depositor can recover its setup capital and steal up to 25% of the next sUSD3 deposit or recapitalization. This is direct victim asset loss and leaves the junior first-loss buffer short by the stolen amount, reducing senior protection and junior-backed credit capacity. Unlike the corresponding USD3 path, sUSD3 can enter the one-share state at initialization and needs only a single intended fee-free report/unlock.

RECOMMENDATION

Add virtual assets/shares or permanently locked dead shares with a sufficient decimal offset and enforce a meaningful minimum initial sUSD3 deposit. Exclude unsolicited donations from reportable assets or provide a safe empty-vault recovery/migration path.

RESOLUTION

Status: Acknowledged.

LOW L-20 ACKNOWLEDGED

Latent borrower premiums let fresh deposits bypass deployment and debt caps.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/libraries/periphery/MorphoBalancesLib.sol:expectedMarketBalances():L33-L60; packages/moneymarket-contracts/src/libraries/periphery/MorphoBalancesLib.sol:expectedSupplyAssets():L94-L104; packages/moneyma ...

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 sizes a lasting new deposit's Morpho deployment from expected market balances that project aggregate IRM interest but omit economically accrued borrower-specific DRP/IRP until each borrower is touched. A second credited borrower can therefore consume cash deployed against an understated supply position while aggregate debt is likewise understated; crystallizing another borrower's premium afterward reveals both exposure ceilings were exceeded.

Impact: Permissionless borrower-touch ordering can create principal exposure beyond both MAX_ON_CREDIT and DEBT_CAP, and tend cannot recall cash that has already been borrowed. The controlled PoC demonstrates over 400,000 USDC of incremental principal extraction and over 333,000 USDC of additional realized loss to unrelated senior depositors after identical defaults, settlements, and reports.

RECOMMENDATION

Use the same fully accrued borrower-premium state for market-wide deployment sizing and debt-headroom checks. Accrue all indebted borrowers before these operations, or maintain a continuously updated aggregate premium/penalty receivable that both USD3 and MorphoCredit caps include.

RESOLUTION

Status: Acknowledged.

LOW L-21 RESOLVED

Matured exit claims accrue irrelevant replay debt from later calls.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/lcc/libraries/LCCAccountLib.sol:isZeroExposure():L50-L53; packages/moneymarket-contracts/src/lcc/LCCVault.sol:_replayAccount():L1138-L1209

REVIEW Round 1 - Main Review

DESCRIPTION

Once an exit has matured, the account has no active or pending callable exposure, but its nonzero claimableExitMargin makes isZeroExposure() false. Consequently _replayAccount() processes every subsequently finalized call instead of fast-forwarding the cursor. Update entrypoints can replay only 64 calls, and claimExitedMargin() reverts when replay remains incomplete, rolling back that call's attempted progress. This is distinct from the prior zero-value wind-down cleanup issue, whose operative defect is claimRemainingMargin() reverting on NothingToClaim.

Impact: Unrelated calls finalized after exit maturity can delay access to already risk-free claimable margin. Each additional 64 calls requires another materializeAccount transaction before claiming, potentially making small or old claims uneconomic, though the funds remain eventually recoverable permissionlessly.

RECOMMENDATION

Add a fast path based on absence of callable exposure rather than absence of all balances. When an exit is matured and active/pending margin and commitment are zero, retain claimableExitMargin and exit metadata while advancing calledEpochCursor to finalizedCallPrefix.

RESOLUTION

Status: Resolved.

Fixed in https://github.com/3jane-protocol/moneymarket-contracts/pull/136/changes/963dd49094648c02f5f811995f4ea0b880eba678.

LOW L-22 ACKNOWLEDGED

Pre-report tranche switching captures historical junior fees.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:syncTrancheShare():L793-L820; packages/moneymarket-contracts/lib/tokenized-strategy/src/TokenizedStrategy.sol:report():L1109-L1183; packages/moneymarket-contracts/src/usd3/sUSD3.sol:_preDe...

REVIEW Round 1 - Main Review

DESCRIPTION

USD3 allocates interval junior compensation by minting performance-fee shares to sUSD3 at report time, without checkpointing which sUSD3 holders supplied first-loss capital while the profit accrued. A direct USD3 holder can move already-owned shares into sUSD3 immediately before a predictable report and receive a pro-rata share of the entire historical junior fee. The isolated PoC compares identical branches after one year of real Morpho interest: leaving 500,000 USD3 direct versus switching only those shares into sUSD3 immediately before report. No post-accrual USD3 issuance or post-report sUSD3 deposit occurs. Profit and fee minting remain identical, yet the switcher captures five-sixths of the fee while the long-term 100,000-USD3 junior loses the same value.

Impact: Point-in-time liquidity can appropriate most compensation intended for capital that bore historical first-loss risk. In the PoC the switcher captures 83.33% of the interval fee; capture approaches 100% as its position approaches available junior capacity. Although the attacker accepts the normal lock, cooldown, and future junior risk after switching, it receives compensation for a completed interval during which it provided no first-loss backing. Predictable or mempool-visible reports make the strategy repeatable and dilute persistent junior underwriters' returns.

RECOMMENDATION

Attribute tranche rewards through a cumulative reward index/reward-debt checkpoint over sUSD3 balances, or otherwise time-weight junior balances across each accrual interval, rather than assigning the entire interval fee to ownership at report time.

If that complexity is not justified, limit exposure through frequent reporting and private report submission. These operational controls only reduce the profitable capture window; they do not eliminate the underlying point-in-time allocation issue.

RESOLUTION

Status: Acknowledged.

LOW L-23 ACKNOWLEDGED

Stale waUSDC capacity quote makes USD3 repeatedly retry an unexecutable exact maximum.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:_wrapUSDC():L216-L232; packages/onchain-ethereum-type4/src/__onchain/statatokenv2-487c2c53c0866f0a/src/contracts/extensions/stata-token/ERC4626StataTokenUpgradeable.sol:maxMint():L143-L147...

REVIEW Round 1 - Main Review

DESCRIPTION

waUSDC quotes positive supply-cap headroom using stored accruedToTreasury, but Aave realizes additional pending treasury accrual before validating a state-changing supply. USD3 selects the exact stale maximum, catches its cap failure, and does not retry a smaller executable amount. Reversion rolls back the treasury checkpoint, so deposit, tend, and report preserve and repeat the unusable quote until an unrelated successful Aave reserve action occurs.

Impact: Already-existing positive waUSDC capacity remains inaccessible to USD3, leaving accepted USDC temporarily idle, yieldless, and unavailable to Morpho borrowers. Principal remains safe, and unrelated Aave reserve activity can restore wrapping, limiting severity to Low.

RECOMMENDATION

Treat maxMint as a fallible boundary: retry a conservatively smaller amount or bounded-search for executable capacity, emit failed-wrap telemetry, and signal loose USDC to maintenance.

RESOLUTION

Status: Acknowledged.

LOW L-24 ACKNOWLEDGED

Yearn protocol fee reduces the configured sUSD3 yield share.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/USD3.sol:syncTrancheShare():L793-L820; packages/moneymarket-contracts/lib/tokenized-strategy/src/TokenizedStrategy.sol:report():L1124-L1161

REVIEW Round 1 - Main Review

DESCRIPTION

syncTrancheShare() writes the configured yield share directly as TokenizedStrategy's gross performance fee. On profitable reports, TokenizedStrategy deducts the factory-level protocol fee from that amount before minting the remainder to the configured sUSD3 fee recipient.

Impact: If the immutable Yearn factory enables a protocol fee, junior lenders receive less yield than the protocol's configured tranche share. For example, a 30% tranche share and 20% factory cut deliver only 24% of profit to sUSD3.

RECOMMENDATION

Either route the junior allocation outside Yearn's fee split, gross up the TokenizedStrategy performance fee so the intended net junior share is received, or explicitly define and document `TRANCHE_SHARE_VARIANT` as a gross share. If gross-up is used, derive it from the current factory protocol fee, resynchronize it before the next profitable report after any fee change, and reject configurations that cannot be satisfied—most notably a desired 100% net junior share with any nonzero protocol fee. Update the tests to assert the selected net or gross semantics.

RESOLUTION

Status: Acknowledged.

LOW L-25 ACKNOWLEDGED

Direct USD3 transfers let fresh junior capital evade its lock and shift default losses to seniors.

CATEGORY Informational

LOCATION packages/moneymarket-contracts/src/usd3/sUSD3.sol:_preDepositHook():L132-L151; packages/moneymarket-contracts/src/usd3/sUSD3.sol:_harvestAndReport():L119-L125; packages/moneymarket-contracts/src/usd3/sUSD3.sol:availableWithdrawLimit():L2...

REVIEW Round 1 - Main Review

DESCRIPTION

sUSD3 refreshes lockedUntil and mints freshly locked shares only when capital enters through deposit or mint. A mature holder can instead transfer USD3 directly to sUSD3, creating no new sUSD3 shares and no new lock while the raw balance immediately counts as junior backing. A keeper report recognizes the transfer as profit of the seasoned share supply. Once Yearn profit unlocking completes, those old shares can redeem the value above the live debt-backing floor through their existing lock/cooldown eligibility; the equivalent ordinary deposit would remain locked for 90 days.

Impact: Fresh first-loss capital can leave before its intended exposure period and shift an equal portion of a subsequent default to seniors. The updated passing PoC retains live 10,000-USDC debt, a 20% ratio floor, a 2,100-USDC nominal floor, and USD3's automatic deployment recall. About 2,900 USD3 above the floor nevertheless exits to USDC before 90 days. After zero-cover settlement and reporting, the independent senior's claim is about 2,900 USDC lower than in the identical control where the 4,000 USD3 entered via deposit and remained locked.

RECOMMENDATION

Before enabling a nonzero deposit lock, restrict all user controlled USD3 credits to sUSD3, including direct transfers and USD3 deposits or mints naming sUSD3 as receiver, so junior capital must enter through the normal sUSD3 deposit flow and receive the configured lock. If direct protocol recapitalization is required, provide an authorized path that preserves equivalent lock or quarantine semantics.

Given the limited likelihood and multi-step prerequisites, consider whether this issue is worth fixing before the next scheduled upgrade; otherwise, explicitly accept it as a low-severity residual risk.

RESOLUTION

Status: Acknowledged.

LOW L-26 ACKNOWLEDGED

Locked profit bypasses junior loss waterfall.

CATEGORY Logical Error LOCATION USD3.sol:583-601 REVIEW Round 1 - Main Review

DESCRIPTION

USD3 states that losses are absorbed by the junior sUSD3 tranche first. `_postReportHook` reverses that ordering whenever locked profit covers part or all of a loss. It computes the junior shares needed at the pre-report price, subtracts `lockedBurn` representing strategy-owned locked-profit shares already burned, and charges sUSD3 only the remainder (`USD3.sol:583-592`). When the buffer covers the loss, senior yield absorbs it and the junior burns nothing.

The accounting does not record which tranche actually paid. If the markdown later reverses, the recovery arrives as ordinary profit and `USD3.report` applies the standing tranche share. The junior is therefore minted a share of a restitution for a loss it never absorbed.

This converts the initial waterfall violation into a permanent transfer rather than temporary report timing. With 8,240,000 USDC senior and 1,100,000 USDC junior, a 136,038 USDC markdown followed by full reversal leaves the senior claim 24,649 USDC below its starting value and junior 24,649 USDC above it, despite no junior shares being burned. Zeroing the tranche share during the sequence lands on the no-transfer baseline.

The live profit-unlock window is three days, so ordinary keeper reports can encounter a nonzero locked-profit buffer. The same missing attribution also matters whenever the junior burn is capped below the loss actually assigned to it.

RECOMMENDATION

Enforce the stated waterfall. Charge sUSD3 for the loss first, up to the junior balance, and apply locked profit only to the residual that the junior cannot cover.

Also record the amount each tranche actually absorbed. Maintain a loss-carry accumulator for loss not borne by junior principal, including amounts displaced by locked profit or the junior-balance cap. On a later recovery, restore that carry to the senior side before applying the standing tranche share, and split only the remaining profit.

Add a test that reports a loss while locked profit is nonzero, fully reverses the markdown, and asserts that both senior and junior claims return to their pre-loss values without transferring value between tranches.

RESOLUTION

Status: Acknowledged.

MEDIUM M-01 RESOLVED

Dust repayment backdates premium to a later draw.

CATEGORY Logical Error LOCATION MorphoCredit.sol:302-303 REVIEW Round 2 - Remediation Review

DESCRIPTION

MorphoCredit computes premium from the recorded debt and the time premium was last accrued; both must describe the same instant, or an old period is charged against recently borrowed debt.

When a premium-bearing borrower is repaid down to a tiny residual, premium on that dust falls below `MIN_PREMIUM_THRESHOLD` and `_accrueBorrowerPremium` returns without advancing the accrual time (`MorphoCredit.sol:301-304`). The timestamp goes stale while the dust sits there.

If the borrower later draws a large loan, `_snapshotBorrowerPosition` records the new debt but keeps the old timestamp (`:321-340`). The next accrual charges that loan for the whole period in which only dust existed, fabricating premium the borrower never incurred and crediting it to lender assets.

Nothing rejects the dust: `MIN_BORROW` is 0 on mainnet, so the check never runs, and where it is nonzero repaying by shares rounds up to assets, so repaying all but one share equals the recorded debt and the guard sees no shortfall. `repay` accepts any `onBehalf`, so a third party can arm another borrower's clock.

The accepted finding "Backdated zero-to-nonzero DRP creates phantom yield" reached the same mismatch via a rate change. The remediation updates the timestamp on a rate change but not when dust premium falls below the threshold, so the state is still reachable without one, leaving that finding only partially remediated.

RECOMMENDATION

Re-stamp the borrower's premium accrual time whenever their recorded principal grows, in the position snapshot taken after a borrow. Premium owed on the old principal has already been charged by that point, so this forfeits only a sub-threshold remainder, and it stops a new, larger principal from being measured against a clock left stale by dust.

Do not instead advance the clock on the below-threshold return inside premium accrual. That closes this path too, but it breaks the guarantee your own accrual-spam test pins — that a sub-threshold accrual must not advance the clock — and reintroduces the yield suppression that test exists to prevent.

The minimum-borrow guard on repayment is a separate layer, and it is currently inert: the value is zero in the live configuration, so the remaining-debt check never runs. To rely on it, set a nonzero minimum and measure it against the debt remaining after the borrow shares are removed. Repaying by shares rounds up, so the pre-repayment figure reads as full repayment and skips the check.

MEDIUM M-01 CONTINUED 2/2 RESOLVED

Dust repayment backdates premium to a later draw.

RESOLUTION

Status: Resolved.

MEDIUM M-02 ACKNOWLEDGED

Gross Interest on Marked Debt Becomes Yield.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L324-L338 REVIEW Round 2 - Remediation Review

DESCRIPTION

Morpho accrues base interest on a borrower's gross face debt and credits the same amount to supplier assets. Borrower markdowns, however, are stored as absolute amounts and increase only when the specific borrower is refreshed. After a borrower has already been materially or fully marked down, later interest therefore increases expected supplier assets while the existing markdown remains fixed. USD3.report() consumes those expected balances without first refreshing every marked borrower, allowing the impaired portion of that interest to be recognized as USD3 profit. For example, assume a $10 million debt has been fully marked down. At 10% APR, it accrues approximately $19,178 of base interest over seven days. Reporting before refreshing the borrower recognizes approximately $19,178 of profit. Refreshing the borrower first instead increases its markdown by the same amount and produces no profit. Once reported, the apparent profit can unlock to USD3 holders and allocate fee value to sUSD3. Holders may withdraw against performing liquidity before the borrower is refreshed. The later markdown increase records the corresponding loss but cannot claw value back from holders who already exited, shifting it to remaining junior and potentially senior holders. The path is inactive while markdown remains disabled. Enabling markdown for a material borrower re-arms it; the disabled setting is an operational control rather than a code-level fix.

RECOMMENDATION

No code change is necessary for the current deployment while markdown remains disabled. Add this behavior to the existing markdown activation checklist and operational risk documentation. If markdown is enabled later, refresh known marked borrowers before USD3 reports where operationally practical, and reassess whether impaired interest requires separate accounting if the resulting exposure becomes material.

RESOLUTION

Status: Acknowledged.

LOW L-01 ACKNOWLEDGED

Recapitalization hides later settlement losses.

CATEGORY Logical Error LOCATION MorphoCredit.sol:917-930 REVIEW Round 2 - Remediation Review

DESCRIPTION

MorphoCredit has a safety floor that stops lender shares falling below a minimum price. The assets it protects scale with the lender shares outstanding.

The defect appears after a large loss leaves the market a few wei above that floor. The floor was not reached, so deposits stay open. A recapitalizing depositor supplies at the depressed share price, receives a very large number of shares, and those shares raise the floor until it sits near the market's whole asset balance.

If another borrower is then settled at a loss, `_applySettlement` subtracts it from `totalSupplyAssets`; when the result would fall below the raised floor it writes the floor back instead (`MorphoCredit.sol:917-930`). The difference is backed by neither tokens nor borrower debt. It is a real loss the accounting has erased, and the same branch latches the market into wind-down.

USD3 reads that balance into its NAV, so it reports value that no longer exists, the loss skips the junior-loss waterfall, and withdrawal limits promise liquidity Morpho cannot return.

The accepted finding "Exact-floor settlement forges USD3 NAV" described this with the first loss landing exactly on the floor, and the remediation closes deposits in that case. A market a few wei above stays unprotected, so it can still be recapitalized and its next loss erased. This is that finding's surviving form.

RECOMMENDATION

Do not remove the share-price floor. It bounds supply shares per unit of assets, and letting the balance fall past it lets the next deposit mint an unbounded share count — your own two-cycle overflow test pins exactly that. Recording the full loss in place is not available.

Record what the floor hides instead. When the clamp fires, accumulate the difference between the loss and the amount actually recorded into a per-market total, and leave the wind-down latch as it is. That change on its own preserves every existing behavior.

Then subtract that accumulated shortfall wherever the market's assets are read as real value — USD3's NAV, and the withdrawal limits derived from it. Those figures currently include value backed by neither tokens nor borrower debt, which is what lets the market report assets it does not hold and advertise liquidity it cannot return.

Protecting the share price and recording the loss stop being in conflict once the unrecorded amount is disclosed rather than erased.

LOW L-01 CONTINUED 2/2 ACKNOWLEDGED

Recapitalization hides later settlement losses.

RESOLUTION

Status: Acknowledged.

LOW L-02 ACKNOWLEDGED

Disabled Markdown Leaves Stale Exit Window.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L463-L562 REVIEW Round 2 - Remediation Review

DESCRIPTION

A borrower becomes Default automatically after its grace and delinquency periods. However, when markdown is disabled—for example, because the market's markdown manager is unset—_updateBorrowerMarkdown() does not impair the debt. Morpho and USD3 therefore continue valuing its principal at par until privileged settlement. USD3 does not inspect borrower status when pricing deposits or withdrawals. Its pending-loss guard activates only after settlement reduces live NAV. During the public Default-to-settlement interval, holders can consequently redeem at the unimpaired price, while new deposits can supply liquidity for those exits. If the account is later written off, locked profit and junior capital absorb what they can, while any uncovered loss falls on the holders who remain. Merged M-07 reported an enabled markdown that was due but had not been materialized. This path is distinct: with the manager unset, no borrower touch can recognize the impairment; settlement is the first accounting event that records the principal loss. Atomic settlement and vault reports close the post-settlement interval but not this earlier servicing window. Default does not guarantee zero recovery, so automatically writing every defaulted loan to zero would be inappropriate. The risk is leaving entry and exit entirely at par after public Default, making loss allocation depend on holder timing.

RECOMMENDATION

Extend the existing settlement runbook to monitor borrowers approaching Default and minimize any delay once impairment is expected. Settle the account and report USD3 and sUSD3 through the existing private atomic bundle. If settlement must be delayed, use the existing USD3 supply-cap and redemption-floor controls to restrict entry and exit, after confirming that no open LCC call depends on USD3 deposits. Treat this servicing procedure as a prerequisite before issuing new third-party credit or posting material repayment obligations.

RESOLUTION

Status: Acknowledged.

LOW L-03 ACKNOWLEDGED

Fee Preview Leaves Deployment Above Cap.

CATEGORY Logical Error LOCATION src/usd3/USD3.sol:L357-L435 REVIEW Round 2 - Remediation Review

DESCRIPTION

USD3.suppliedWaUSDC() uses Morpho's expectedSupplyAssets() to size deployment-cap transitions. That helper explicitly warns that its result is incorrect for the Morpho fee recipient: it projects fee shares from pending base interest into the market's total supply shares but does not add those shares to the fee recipient's projected balance. When USD3 is the fee recipient and the market fee is nonzero, the view therefore understates USD3's position before a stateful accrual. After an sUSD3 exit reduces junior backing, USD3._postTransferHook() calls _applyDeployCap(false). The function calculates the required recall from the understated position. _withdrawFromMorpho() then accrues Morpho interest, which mints the omitted fee shares to USD3, but withdraws only the amount calculated before accrual. USD3 can consequently finish above the reduced deployment target by up to the value of the pending fee shares. The same root affects supply-increasing transitions. Post-deposit _deployDepositedFunds() and keeper-driven _applyDeployCap(true) calculate headroom from the understated position. Morpho's supply() then accrues the missing fee shares before accepting the stale-sized supply, so the final position can exceed a binding target. These are two manifestations of one preview-before-accrual defect. If an authorized borrower consumes the residual before a later tend or report corrects it, that liquidity cannot be recalled. A subsequent default can expose senior capital beyond the amount intended by the junior-backed deployment target. The discrepancy is bounded by pending base-interest fee accrual, and the last verified production settings suppress the path: the Morpho fee is zero, MIN_SUSD3_BACKING_RATIO is zero, MAX_ON_CREDIT is 100%, and no adverse third-party borrower has drawable headroom. The issue becomes loss-relevant only when USD3 is the Morpho fee recipient and a nonzero fee, binding junior-backed deployment cap, and drawable borrower headroom coexist.

RECOMMENDATION

Use a fee-recipient-aware post-accrual position when enforcing the deployment cap. Prefer correcting the preview to include USD3's projected fee shares. Alternatively, accrue as part of the rebalance and recompute the position before determining any supply or recall amount. Do not introduce a new mandatory Morpho accrual on deposits or exits that would not otherwise touch Morpho; if optional rebalancing fails, defer it through the existing best-effort handling. Treat this correction as a prerequisite before combining a nonzero Morpho fee, a binding deployment cap, and drawable third-party credit.

RESOLUTION

Status: Acknowledged.

LOW L-04 ACKNOWLEDGED

Minimum Credit-Line Check Blocks Emergency Revocation.

CATEGORY Logical Error LOCATION src/CreditLine.sol:L136-L159 REVIEW Round 2 - Remediation Review

DESCRIPTION

EmergencyController.emergencyRevokeCreditLine() is intended to stop further borrowing by setting a borrower's credit authorization to zero while preserving the premium rate on existing debt. The controller forwards this update through CreditLine.setCreditLines(), which requires every credit amount to be at least minCreditLine. Whenever minCreditLine is positive, the zero-credit update reverts with MinCreditLineExceeded, preventing selective emergency revocation.

RECOMMENDATION

Add a dedicated privileged revocation path that permits setting credit to zero without applying ordinary minimum-line or underwriting-proof requirements while preserving the borrower's existing premium rate. Alternatively, narrowly exempt authorized zero-credit updates from those admission checks.

RESOLUTION

Status: Acknowledged.

LOW L-05 ACKNOWLEDGED

Minimum Debt Check Prevents Terminal Insurance Use.

CATEGORY Logical Error LOCATION src/CreditLine.sol:L204-L225 REVIEW Round 2 - Remediation Review

DESCRIPTION

CreditLine.settle() applies insurance coverage through ordinary Morpho.repay() immediately before settleAccount() writes off all remaining borrower debt. However, _beforeRepay() rejects any partial repayment that leaves a nonzero balance below MIN_BORROW. Consequently, when 0 < debt − cover < MIN_BORROW, terminal settlement cannot apply the selected insurance coverage even though the remaining debt would be erased immediately afterward. The operator must reduce coverage until at least MIN_BORROW is written off, obtain enough coverage to repay the debt completely, or first change the configuration. Reducing coverage increases the affected settlement's immediate supplier write-off by up to almost MIN_BORROW, potentially passing a larger loss through USD3's loss waterfall. The unused insurance remains in the shared fund but no longer protects this borrower or the holder cohort exposed at settlement. The path is inactive while MIN_BORROW is zero but becomes reachable under the supported positive configuration.

RECOMMENDATION

Before enabling a positive MIN_BORROW, exempt settlement-only insurance repayments initiated by the market's registered CreditLine from the remaining-debt minimum.

RESOLUTION

Status: Acknowledged.

INFO I-01 RESOLVED

Activation gates live only in NatSpec.

CATEGORY Documentation LOCATION MorphoCredit.sol:705-710 REVIEW Round 2 - Remediation Review

DESCRIPTION

Enabling markdown takes two owner actions. The market's `CreditLine` must name a markdown manager, and the owner must set `markdownEnabled` for a borrower. While the manager is unset, `_updateBorrowerMarkdown` returns before touching state (`MorphoCredit.sol:705`), so a group of accepted findings is dormant by configuration and is restored by a single `setMm` call.

The commit records what to remediate before that call: stale deposit pricing, withdrawal freezes, cure misattribution, tranche fee-share sizing, and deployment accounting that excludes markdown. It records this only in a NatSpec comment on that function and in a developer-facing file.

The operations runbook has no markdown-activation entry; its one markdown section covers operating while the feature is off. That runbook already handles a narrower coupling correctly, requiring `DEBT_CAP` to be resized whenever the deployment target drops below full. An operator checking it before enabling markdown would find no checklist.

RECOMMENDATION

Add a "Before enabling markdown" section to the operations runbook in the same form as the existing deployment-cap coupling rule: the two gates that arm the feature, the five items to remediate first, and an instruction not to name the manager until each is closed. The content already exists in the NatSpec, so this is a move rather than new analysis. Apply the same treatment to the other single-write re-arms the acceptances rest on — issuing a third-party credit line, setting a nonzero subordination backing ratio, and setting a nonzero junior lock duration.

RESOLUTION

Status: Resolved.

INFO I-02 ACKNOWLEDGED

Aave index cast can freeze the market.

CATEGORY DoS LOCATION AdaptiveCurveIrm.sol:261 REVIEW Round 2 - Remediation Review

DESCRIPTION

`AdaptiveCurveIrm._updateAaveIndices` stores Aave's normalized indices in `uint96` fields (`AdaptiveCurveIrm.sol:261`). At an index near 1.24 RAY that cast has roughly 64x of headroom. Past it the cast truncates silently, and the stored checkpoint lands far below the live index.

`_calculateAaveSpread` then divides the live index by that checkpoint (`AdaptiveCurveIrm.sol:242`) and hands the ratio to `MathLib.wInverseTaylorCompounded` , a three-term series that subtracts its second term from its first under checked arithmetic. Above a ratio of three the subtraction reverts.

`borrowRate` runs inside `Morpho._accrueInterest` , so a revert there stops the market: supply, borrow, repay and liquidate all fail, and `borrowRateView` fails with them. Nothing clears the state, because the only writer of the checkpoint is the call that now reverts.

RECOMMENDATION

Bound the ratio before the series first: clamp the growth factor at `3 * WAD` , or return a zero spread above it, so an extreme reading degrades the quote instead of reverting. A revert inside `borrowRate` is not a safe failure mode, since Morpho calls it on every state-changing path. This alone removes the brick, and it needs no storage change.

For the truncation itself, prefer storing the index scaled down, which keeps the current layout. Widening the two fields to `uint128` is arithmetically correct but pushes the struct from one slot to two, and it is the value type of a mapping on an upgradeable contract — existing per-market checkpoints would be misread after the upgrade unless they are migrated or moved to a fresh mapping. The eight reserved slots are available if you take that route.

Separately, confirm the deployed rate model matches this source. It has no deployment script in the repository.

RESOLUTION

Status: Acknowledged.

INFO I-03 RESOLVED

Vault creation does not bind oracle to asset.

CATEGORY Validation LOCATION CreateLCCVaultSafe.s.sol:257-260 REVIEW Round 2 - Remediation Review

DESCRIPTION

Creating a facility takes two addresses from a config file: the collateral asset, and the price oracle for it. Nothing checks they belong together. Vault creation confirms only that the oracle address holds code and returns a non-zero price. The thorough oracle validation — internal wiring, and an expected price range — runs in a separate script at deployment time, and checks each oracle on its own, never against the asset it will later be paired with.

The four shipped oracles differ from one another by a factor of a trillion, because some price six-decimal assets and others eighteen-decimal. That spread is deliberate and each oracle is correct on its own. It also means every one of them satisfies the creation checks for every collateral asset, so a mismatched pair is accepted silently and the facility values its collateral at the wrong scale from the first deposit onward. A vault's collateral asset cannot be changed after creation.

Reaching this state requires an operator to mis-enter one of two adjacent fields on a multisig deployment whose resulting vault address is previewed before execution. There is no attacker-controlled path, which is why it is raised as informational.

RECOMMENDATION

Add an expected price range to the vault deployment config, next to the collateral and oracle addresses, and have vault creation reject an oracle priced outside it. The oracle deployment script already does this per oracle, so both the values and the pattern exist — this extends the check to where the two are paired, and catches the decimal mismatch above.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.